Leipzig University
Institute of Computer Science
Degree Programme Computer Science, B.Sc.

Adversarial and Multi-Agent Large Language Models for Code Vulnerability Detection: A Replication and Extension Study

Bachelor's Thesis

Elias Masur

1. Referee: Prof. Dr. Andreas Maletti

Submission date: September 30, 2026

Declaration

I confirm that I have written the bachelor's thesis with the topic: "Adversarial and Multi-Agent Large Language Models for Code Vulnerability Detection: A Replication and Extension Study" independently and have not used any sources or aids other than those specified. I have not submitted this work to any other examination office in the same or a comparable form. It has not been published so far.

AI Usage Disclosure:
I acknowledge that AI-based tools, namely Claude and Grammarly, were used to support literature exploration, software development, and language refinement. The author of this thesis critically reviewed and validated all outputs.

Leipzig, September 30, 2026

. . . . . .
Elias Masur

Abstract

Large language models (LLMs) are increasingly proposed for detecting vulnerabilities in source code. However, reproducing reported comparisons is challenging when data and prompts are not publicly available. Saju et al. evaluate four LLM-based approaches: direct prompting (the baseline), retrieval-augmented generation (RAG), supervised fine-tuning (SFT), and a Dual-Agent system, reporting RAG as the strongest approach on a partly proprietary corpus. This thesis replicates the comparison with the same model, Llama 3.2 with three billion parameters, on the public Big-Vul corpus alone and under three random seeds. Pairwise differences on individual predictions are tested using McNemar's test. The previously reported ordering does not hold under these conditions. The fine-tuned configuration achieves the highest macro-F1 (0.69 compared to 0.54 for RAG) and significantly outperforms RAG in every seed. The Dual-Agent system (0.47) falls behind the baseline (0.52), though not significantly. Only the fine-tuned configuration improves on the baseline significantly, and not after correction for multiple testing. Among the prompting approaches, RAG's lead over the baseline lies in how often it answers *Vulnerable* rather than in how well it distinguishes vulnerable from safe code, whereas the Dual-Agent system loses that distinction at an unchanged rate. As an extension, adversarial prompting from LLM-GAN is transferred to code. One frozen model acts as Generator (rewriting clean functions to introduce a weakness of an assigned class), as Detector, and as Reflector, with roles updated through textual strategy memories instead of gradients. The central result of this extension is negative and undermines its premise. A manual check of 30 generated functions found none that exhibit the intended weakness. Robustness could therefore not be assessed, and the final Detector's differences from the prompting approaches cannot be attributed to adversarial learning. A Triple-Agent setup, with the Dual-Agent system in place of the single Detector, is correct less often than the final Detector in every seed, though not significantly. A reproducibility audit shows that the gaps no third party can close concern the data: the proprietary part of the corpus, the curation of the splits, the expert review of the labels, and the documents behind the RAG knowledge base. In summary, this thesis provides a public, statistically tested replication, a negative result for adversarial prompting on code, and an inventory of the gaps that bound what any public replication can compare.

Contents

List of Acronyms **IV**

1 Introduction **1**
1.1 Motivation 1
1.2 Problem Statement 2
1.3 Research Questions 3
1.4 Contributions and Scope 3
1.5 Thesis Outline 4

2 Background **5**
2.1 Software Vulnerabilities and the CWE 5
2.2 Large Language Models and Instruction Following 5
2.3 Retrieval-Augmented Generation 6
2.4 Parameter-Efficient Fine-Tuning 7
2.5 Multi-Agent LLM Systems 8
2.6 Adversarial Paradigms 8
2.7 Evaluation and Statistical Testing 9

3 Related Work **11**
3.1 LLM-Based Vulnerability Detection 11
3.2 Self-Improving and Multi-Agent LLMs 11
3.3 Adversarial Prompting and Code Generation 12
3.4 Vulnerability Datasets 13
3.5 Research Gap 14

4 Replicated Detection Approaches **15**
4.1 Replication Protocol 15
4.2 Baseline (Direct Prompting) 17
4.3 Retrieval-Augmented Generation 17
4.4 Supervised Fine-Tuning 18
4.5 Dual-Agent System 19
4.6 Reproducibility Audit 19

5 Adversarial Extension **21**
5.1 From LLM-GAN to Code . . . 21
5.2 The Generator . . . 22
5.3 Detector, Reflector and Strategy Memories . . . 23
5.4 The Adversarial Loop . . . 24
5.5 Validity Checks . . . 25
5.6 Triple-Agent Extension . . . 26

6 Experimental Setup **27**
6.1 Data . . . 27
6.2 Model, Environment and Reproducibility . . . 29
6.3 Per-Approach Configuration . . . 29
6.4 Evaluation Metrics . . . 30
6.5 Statistical Analysis . . . 31

7 Results **33**
7.1 Overview . . . 33
7.2 Replication Results (RQ1) . . . 34
7.2.1 Observed Ranking vs. the Base Study . . . 34
7.2.2 Per-Approach Performance . . . 36
7.3 Adversarial Extension Results (RQ2) . . . 37
7.3.1 Generated Functions and Setup Checks . . . 37
7.3.2 Detection Performance vs. Baseline and Dual-Agent . . . 39
7.4 Triple-Agent Results (RQ3) . . . 40
7.5 Cost and Efficiency . . . 42
7.6 Qualitative Error Analysis . . . 43

8 Discussion **45**
8.1 Answering the Research Questions . . . 45
8.2 Why the Order Differs . . . 47
8.3 What the Adversarial Loop Learned . . . 49
8.4 Implications and Practical Feasibility . . . 50
8.5 Threats to Validity . . . 51
8.5.1 Construct Validity . . . 52
8.5.2 Internal Validity . . . 52
8.5.3 Statistical Conclusion Validity . . . 53
8.5.4 External Validity . . . 53

9 Conclusion and Future Work **54**
9.1 Summary of Contributions . . . 54
9.2 Future Work . . . 55

A Full Prompts **57**
A.1 Role Prompts of the Extension 57
A.2 Final Strategy Memories 60

B Author Correspondence **63**

Bibliography **67**

List of Acronyms

CVE Common Vulnerabilities and Exposures

CWE Common Weakness Enumeration

F1 F1 score (harmonic mean of precision and recall)

GAN Generative Adversarial Network

GPU Graphics Processing Unit

HTTPS Hypertext Transfer Protocol Secure

LLM Large Language Model

LoRA Low-Rank Adaptation

MCC Matthews Correlation Coefficient

NVD National Vulnerability Database

PPR Predicted-Positive Rate

QLoRA Quantised Low-Rank Adaptation

RAG Retrieval-Augmented Generation

RQ Research Question

SFT Supervised Fine-Tuning

Chapter 1

Introduction

1.1 Motivation

Software vulnerabilities remain a persistent and costly cause of software failure. The submissions of Common Vulnerabilities and Exposures (CVE) records to the National Vulnerability Database (NVD) rose by 263 % between 2020 and 2025 (NIST, 2026). A single flaw in a widely deployed component can expose every system that relies on it. Following the disclosure of Heartbleed, an estimated 24–55 % of popular HTTPS servers were vulnerable at once, and exploitation attempts had already begun within a day (Durumeric et al., 2014). Since code is written and modified faster than it can be checked manually, automated detection is essential.

In practice, two families of automated detectors are commonly used. Static analysers work by matching code against fixed patterns derived from known vulnerabilities. They require no training data and can scan large codebases efficiently. However, on validated CVEs from real-world C projects, leading analysis tools failed to detect between 47 % and 80 % of the vulnerabilities (Lipp, Banescu, and Pretschner, 2022), and the false positives they generated have long hindered their use (Johnson et al., 2013). By contrast, learning-based detectors derive their decision criteria from datasets of vulnerable and fixed functions and tend to achieve strong benchmark scores. Nevertheless, they output a verdict without any justification, so an evaluator cannot determine whether the model actually took the flaw itself into account or merely an incidental feature that correlates with the label. Four such models were tested on real-world code, and their performance collapsed. Explanation techniques traced their predictions to code elements unrelated to the vulnerability (Chakraborty, Krishna, et al., 2022).

Large language models (LLMs) change what a detector can return by combining a verdict with a natural language justification in a single output. The

justification is no proof and may be constructed post hoc, but it can be inspected and rejected on its content. An LLM-based detector can also be adapted without retraining, by changing its instructions, its retrieved context or its orchestration. Direct prompting, retrieval of security-relevant knowledge, parameter-efficient fine-tuning, and setups in which several model instances assume distinct roles are all configurations of the same base model. Which of them actually improves detection is an empirical question, and the study replicated and extended in this thesis is an attempt to answer it.

1.2 Problem Statement

Saju, Muhtadi, and Azim (2026) assess four configurations of a small open-weight model, Llama 3.2 with three billion parameters in the instruct variant (hereafter "the base model"). They apply it to vulnerability detection framed as binary classification over code changes. Each instance contains a function, its version after a fixing commit, the corresponding commit message, and metadata such as the CVE and Common Weakness Enumeration (CWE) identifiers. The label indicates whether the function was vulnerable before the change. The four configurations are direct prompting as the baseline, retrieval-augmented generation (RAG), supervised fine-tuning (SFT), and a Dual-Agent system in which a Validator reviews the Detector's verdict. According to the study, RAG outperforms SFT, the Dual-Agent system, and the baseline. This ranking is based on mean F1 scores. Yet each technique is tested only against the baseline using a paired t-test across five CWE categories. That means the ordering among the three techniques themselves rests on no reported test.

The study leaves two further gaps. The first concerns the data. Part of the study's corpus is a proprietary industrial collection, so the curated dataset cannot be released. It is clear from the correspondence with the authors (Appendix B) that this part is included in both the training and test datasets. As a result, the reported figures cannot be reproduced by outside parties. The same correspondence also confirms that each configuration's results stem from a single run, so the ranking comes with no estimate of run-to-run variance. What can still be answered is whether the reported ranking holds once the four approaches are re-implemented independently and evaluated on the public Big-Vul corpus (Fan et al., 2020).

The second gap concerns the method space itself. All three techniques aim to improve the detector by providing additional information through retrieved knowledge, gradient updates derived from labelled examples, or a second agent's review. However, none of them pits the detector against an adversary actively trying to defeat it. Wang et al. (2025) show with LLM-GAN for

explainable fake-news detection that such an arrangement can be built through prompting alone. No generative adversarial network (GAN) is trained. Textual feedback takes the place of gradient updates. Whether this approach can be applied to code has not yet been tested and is not obvious. Unlike a generated news item, a generated function is not vulnerable simply because it was generated. Its label must therefore be established, not assumed.

1.3 Research Questions

The replication comes first, because an extension needs a comparison base established in the same pipeline.

RQ1 (Replication). (a) Does the F1-based ranking reported by Saju, Muhtadi, and Azim (2026), RAG > SFT > Dual-Agent > Baseline, hold when all four approaches are independently re-implemented and run on the public Big-Vul corpus alone, and (b) are the differences between them statistically significant?

RQ2 (Adversarial extension). Does a prompting-based adversarial approach, transferred from LLM-GAN to code vulnerability detection, improve over the baseline and the Dual-Agent system in (a) detection performance on the real, held-out test set and (b) robustness?

Robustness refers only to how a detector judges functions that the Generator has modified to introduce an inconspicuous flaw, compared with its verdicts on the clean source functions. Whether a generated function actually contains the intended flaw is validated separately (Section 5.5), and part (b) is conditional on that validation. If generated functions do not carry the intended flaw, part (b) cannot be measured as posed, and the validation result is reported in its place. The baseline and the Dual-Agent system are the comparison partners because they use prompting alone, like the adversarial approach. RAG and SFT are reported for reference.

RQ3 builds on RQ2 and asks whether the detecting side of the adversarial setup benefits from being a two-stage system rather than a single agent.

RQ3 (Triple-Agent). Does employing the full Dual-Agent system (Detector plus Validator) as the detecting party in the adversarial setup, yielding a Generator + Detector + Validator triad, further improve the results over the adversarial setup of RQ2?

1.4 Contributions and Scope

This thesis makes three contributions.

Replication. This thesis re-implements the four approaches from the base study in its own code, using every prompt from the paper or the authors' release. They are evaluated using the public Big-Vul corpus in one pipeline under a controlled setup. Each approach is run under three random seeds, and differences between approaches are tested on individual predictions rather than on five per-category F1 scores.

Adversarial extension. The GAN-inspired adversarial prompting of LLM-GAN is transferred to code vulnerability detection. No network is trained. Generator, Detector and Reflector (Section 2.6) are roles of one frozen model, and what circulates between them is text rather than a gradient. Beyond the specifications of LLM-GAN, this thesis verifies whether the generated functions actually carry the label assigned to them, rather than simply assuming this to be the case. The result of this verification determines which parts of RQ2 can be answered. The thesis also replaces the single Detector by the Dual-Agent system to form a Triple-Agent setup (RQ3).

Reproducibility audit. The thesis inventories what the paper, the released artefacts, and the authors' answers specify about the base study and what they leave open. It also separates gaps that a replication can close by a documented choice from gaps that cannot be closed because the material is not public (Section 4.6). The audit bounds what remains comparable in Chapter 8.

Scope. As only the public Big-Vul corpus is used, generalisation across corpora is not measured. Llama 3.2 with three billion parameters is used in every approach, so nothing here speaks to how the ordering would change with larger or different models. Only the verdicts are evaluated, not the quality of the justifications that accompany them.

1.5 Thesis Outline

Chapter 2 supplies the concepts the thesis relies on. Chapter 3 locates the base study and the extension among neighbouring work. Chapter 4 presents the four replicated approaches and the reproducibility audit, Chapter 5 the adversarial extension. Chapter 6 describes the experimental setup. Chapter 7 reports the results. Chapter 8 interprets them as answers to RQ1–RQ3 and weighs the threats to validity. Chapter 9 summarises the contributions and names what remains open. The appendices hold the prompt texts and final strategy memories of the extension, followed by the author correspondence.

Chapter 2

Background

2.1 Software Vulnerabilities and the CWE

A vulnerability is a specific flaw in a specific version of a program that an attacker can exploit. The Heartbleed bug in OpenSSL (Durumeric et al., 2014) is one example. A weakness is the general type of such a flaw, for example, reading beyond the bounds of a buffer. The Common Weakness Enumeration (CWE) is MITRE's catalogue of those types (MITRE, 2026). Each entry carries a stable numeric identifier, a name, a description, typical consequences, mitigations, and illustrative code. A CVE record names one specific vulnerability. A CWE entry names the class.

Vulnerability datasets such as Big-Vul label each function with a CWE identifier, and the catalogue entries can serve as security knowledge for a detector. Three properties of the catalogue matter for both uses. Identifiers from different levels are not interchangeable, because entries sit at different levels of abstraction and are linked by parent–child relations. The catalogue is versioned, so a dataset and a knowledge base built against different releases may not describe the same categories. Not every identifier denotes a weakness. *Category* entries are thematic groupings with a short summary and a list of members, but without the description, consequence, and example content of a weakness entry. A study that uses CWE labels therefore has to state which release it refers to and at which level of the hierarchy it compares identifiers.

2.2 Large Language Models and Instruction Following

A decoder-only language model defines a probability distribution over the next token based on all preceding tokens (Radford et al., 2019). The models con-

sidered here are transformers (Vaswani et al., 2017). Since text generation is achieved by repeatedly applying this distribution, the instruction, the data, and the required answer format must all be included in the token sequence on which the model is conditioned. This sequence is bounded by a finite context window, which, in the case of Llama 3.2, comprises 128k tokens (Meta, 2024).

Two variants of the same pretrained weights are usually released. The base variant continues whatever text it is given, whereas the instruct variant is additionally post-trained on instruction-following demonstrations and human preference feedback (Ouyang et al., 2022). As a result, "answer with *Vulnerable* or *Not Vulnerable*" becomes a request to follow rather than text to continue. A classification task expressed as an instruction therefore presupposes the instruct variant. A chat template that wraps each turn in role markers and special tokens is also expected by instruct variants (Meta, 2024). The same characters submitted with and without the template are different inputs to the model.

The prompt is a parameter of the experiment, not packaging around the task. Even choices that carry no meaning, such as separators or whitespace, shift measured accuracy (Sclar et al., 2024). Therefore, a prompt must be fixed exactly, and a comparison between two approaches isolates a mechanism only if their prompts differ in the intended respect and nothing else. Decoding is a second such parameter. The decoder can take the most likely token at each step, which makes a run deterministic up to numerical effects of the runtime. Alternatively, it can sample with a temperature and a nucleus threshold (Holtzman et al., 2020). Sampling with a fixed seed is reproducible on the same stack, while different seeds reveal the variation introduced by sampling.

2.3 Retrieval-Augmented Generation

A language model's knowledge is fixed in its weights. Retrieval-augmented generation (RAG) instead supplies task-relevant text during inference. A retriever selects passages from an external corpus and places them in the prompt alongside the task (Lewis et al., 2020). The variant relevant here retrieves only at inference time and leaves the model frozen (Ram et al., 2023), so the corpus can be replaced without retraining. Saju, Muhtadi, and Azim (2026) use a pipeline of four stages, each with parameters of its own.

First, documents are split into *chunks* small enough to be embedded and to fit into a prompt. Size and overlap are specified in characters or in tokens from a tokeniser, so the same number can represent different amounts of text depending on the unit. A bi-encoder maps each chunk to a vector, which is then stored in an index. At query time, the nearest chunk vectors are returned

under a distance function. Encoders of the BGE family are a common open choice (Xiao et al., 2024). A bi-encoder never processes the query and passage together, so its ranking is only approximate. A cross-encoder instead scores each query-passage pair jointly and *re-ranks* a larger set of candidates, reducing it to the few passages that are ultimately included in the prompt (Nogueira and Cho, 2020). Where the selected passages sit in the prompt is another parameter. According to Liu, Lin, et al. (2024), models use information more reliably at the beginning and end of a long input than in the middle. A RAG configuration is defined by the document collection, chunking, the embedding model and distance function, candidate and final set sizes, and the re-ranker and context position.

2.4 Parameter-Efficient Fine-Tuning

Full model fine-tuning updates every weight and requires storing both the gradient and optimiser state for each parameter, resulting in memory usage that is a multiple of the memory occupied by the weights. Parameter-efficient approaches instead freeze the pretrained weights and train only a small set of additional parameters. Low-rank adaptation (LoRA) assumes that the weight changes required for a given task have low intrinsic rank (Hu, Shen, et al., 2022). For the frozen weight matrix W of a linear projection, of shape $d \times k$, it learns an update $\Delta W = BA$, with B of shape $d \times r$, A of shape $r \times k$, and $r \ll \min(d, k)$. The adapted layer computes

$$h = Wx + \frac{\alpha}{r} BAx,$$

where x is the input of the layer, h its output, and α is a fixed scaling constant. Only A and B receive gradients, so the number of trained parameters is governed by r. Which projections receive an adapter and the values of r and α are left open by the method.

QLoRA reduces the memory cost further by quantising the frozen base weights to the four-bit NormalFloat format while keeping the adapters at higher precision (Dettmers et al., 2023). What is trained is thus an adapter fitted to a quantised model. Because ΔW has the shape of W, the adapter can be folded into the base weights after training,

$$W' = W + \frac{\alpha}{r} BA,$$

which yields an ordinary dense model. An adapter fitted to a quantised base can also be merged into the unquantised model, but the evaluated weights are no longer the weights the adapter was optimised against. Every configuration

has two additional settings. The number of examples a run consumes is the number of optimiser steps times the effective batch size. Each example is serialised into a single token sequence of prompt and target, with the loss taken over the whole sequence or only over the target (von Werra et al., 2026).

2.5 Multi-Agent LLM Systems

In this thesis, the term "agent" is used in a narrow sense. An agent refers to a single invocation of a language model under a role-specific prompt, and multiple such invocations, connected within a fixed topology, are referred to as a multi-agent system. In this process, the output of one agent becomes part of the input for the next. The wider literature attaches more to the term, including tool use, persistent memory and autonomy over control flow (He, Treude, and Lo, 2025), but none of that is required here. Roles can share weights, because a role is a prompt rather than a model. One loaded model can serve every role, and adding a role costs one more model call. Whether roles share weights and decoding settings must be stated, since it determines whether a disagreement between two roles is a second opinion or the same model resampled under a different instruction.

The Dual-Agent system of the base study follows a two-stage sequential pattern (Saju, Muhtadi, and Azim, 2026): a first role, the Detector, produces a verdict with its reasoning, and a second role, the Validator, audits that output. The pattern leaves open what the auditing role sees, which of the two verdicts the system reports and how many rounds the roles exchange. Each changes the reported numbers without changing the pattern. How the replication settles them is the subject of Section 4.5.

2.6 Adversarial Paradigms

A generative adversarial network (GAN) consists of two neural networks with opposing objectives (Goodfellow et al., 2014). A generator maps noise to samples, and a discriminator estimates whether a given sample comes from the data or from the generator. Both networks are trained against each other on one minimax objective. The only feedback the generator receives is the discriminator's response, which is propagated as a gradient through the differentiable discriminator. Consequently, both networks must be trainable, and the generated samples must reside in a space through which a gradient can pass. It is precisely this gradient path that cannot be carried over to prompting.

The prompting-based reinterpretation retains the structural arrangement of generator and detector roles but omits the training process (Wang et al.,

2025). In this framework, the Generator and Detector are roles in the sense of Section 2.5, exchanging natural-language explanations rather than gradients. When the Detector is fooled by a generated item, it receives an explanation from the Generator describing how the item was constructed. Conversely, when the Detector succeeds, the Generator receives an explanation detailing how the item was caught. Each role maintains a strategy set, which is initially empty and updated only in the rounds it loses. Additionally, a self-reflection stage without the Generator introduces a third role. The Detector judges labelled training data, the Reflector explains each wrong verdict, and the Detector revises its strategies based on that explanation.

The name suggests more than the mechanism delivers. The feedback exchanged between roles consists of model-generated text rather than gradients derived from a formal objective function. This feedback therefore offers a heuristic and potentially biased basis for strategy revision. Improvement in performance is not guaranteed, and the number of interaction rounds serves as a fixed budget rather than a convergence criterion. Moreover, because the roles share model parameters, the critic and the criticised share their blind spots. "Adversarial" thus refers to the structural arrangement of roles, rather than to an adversarial training procedure.

The naming convention of this thesis follows from the roles. Specifically, "Dual-Agent" refers exclusively to the base study's Detector–Validator system described in Section 2.5. The adversarial setup is a Generator paired with a single Detector. The term "Triple-Agent" denotes an adversarial configuration in which the Dual-Agent system serves as the detecting party, resulting in three roles: Generator, Detector, and Validator, each responsible for producing or evaluating a function. The Reflector, which provides only explanatory feedback, is not counted in either name.

2.7 Evaluation and Statistical Testing

Metrics. A binary detector is scored through the confusion counts of the positive class, here the vulnerable one: true positives (TP), false positives (FP) and false negatives (FN). Precision P is the share of flagged instances that are vulnerable, recall R the share of vulnerable instances that are flagged, and F1 their harmonic mean:

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad \mathrm{F1} = \frac{2PR}{P + R}.$$

When F1 is computed separately for each CWE category and averaged without weights, the result is a macro-F1 that assigns equal importance to each category, regardless of the number of instances per category. The F1 score

for the positive class considers only true positives, false positives and false negatives, and ignores true negatives. For example, on a class-balanced test set, a classifier that predicts *Vulnerable* for every instance achieves $P = 0.5$ and $R = 1$, and therefore an F1 of 0.667, without distinguishing between the classes at all. This matters on balanced data: the metric rewards frequent flagging, and an F1 near 0.667 is compatible with no discrimination between classes. To make this visible, this thesis also reports the Matthews correlation coefficient (MCC), which incorporates all four confusion counts, is 1 for a perfect classifier and is zero, by convention or in expectation, for a classifier that ignores its input (Chicco and Jurman, 2020). In addition, it reports the predicted-positive rate (PPR), defined as the proportion of instances a detector flags as *Vulnerable*. Reporting MCC and PPR alongside F1 exposes the limitation of F1.

Significance tests. Two detectors evaluated on the same instances yield one paired outcome per instance: each of the two is either correct or wrong. McNemar's test (McNemar, 1947) looks only at the discordant instances, those on which exactly one of the two detectors is correct, and asks whether they split evenly between the two. The exact variant used here computes the p-value with a binomial test. It is conservative (Fagerland, Lydersen, and Laake, 2013) and among the tests recommended for comparing two classifiers on a single test set (Dietterich, 1998). Because the test counts errors, it compares accuracy. It does not test F1, so a ranking defined on F1 and a significance statement from McNemar's test concern two different quantities.

Multiple comparisons. How several tests combine depends on the form of the claim. If any one of several significant comparisons would count as a finding, the Holm procedure (Holm, 1979) bounds the probability of at least one false rejection. It compares the smallest of m p-values with α/m, the next with $\alpha/(m-1)$ and so on, stopping at the first comparison that fails. Equivalently, it reports adjusted p-values that are compared with α directly. If instead the claim holds only when every one of several hypotheses is rejected, the intersection-union test (Berger, 1982) applies: the claim is supported only if the largest of the p-values lies below α. For instance, a difference that has to hold under each of three seeds is supported only if all three p-values lie below α. No correction is needed, but every additional hypothesis makes the test more conservative.

Chapter 3

Related Work

3.1 LLM-Based Vulnerability Detection

When grouped by method, the field can be divided into fine-tuning, which adapts the model's parameters, and techniques that leave the weights unchanged, including retrieval, which provides knowledge the model lacks (Sheng et al., 2026). Direct prompting supplies neither and is the reference point. Without additional support, models are not reliable detectors: Ullah et al. (2024) report non-deterministic answers, unfaithful reasoning and verdicts that change when functions or variables are renamed. Retrieval adds knowledge at inference time (Lewis et al., 2020). Vul-RAG retrieves distilled vulnerability knowledge rather than similar code. Du et al. (2025) report clear gains in separating vulnerable from patched code, though on Linux-kernel CVEs and with larger models. Fine-tuning enters in its parameter-efficient form (Section 2.4), which brings the adaptation of models far larger than the one used here within the memory of a single GPU (Dettmers et al., 2023).

The base study compares both axes with direct prompting, alongside the Dual-Agent system of Section 3.2, using one small open model (Saju, Muhtadi, and Azim, 2026). On Llama 3.2 with three billion parameters, across five CWE categories, it reports RAG ahead of SFT, the Dual-Agent system and the baseline, with F1 values of 0.85, 0.80, 0.77 and 0.67.

3.2 Self-Improving and Multi-Agent LLMs

Two patterns recur. The first splits detection across roles, as in the Dual-Agent system replicated here. GPTLens applies a detector–validator pattern to smart contracts, with an AUDITOR proposing candidate vulnerabilities and a CRITIC scoring and ranking them so that only the top-ranked ones are retained (Hu,

Huang, et al., 2023). Mao et al. (2024) let LLMs act as a tester and a developer that discuss a function until they agree on whether it is vulnerable. The base study introduces its Dual-Agent system without relating it to such work. Adding another role is not free. He, Treude, and Lo (2025) count the growing communication complexity among the open challenges, and Huang et al. (2024) argue that multi-agent arrangements should be compared against baselines of comparable inference cost.

The second pattern lets a model revise its behaviour from past mistakes. Reflexion reflects on an external task signal and reinforces the agent through language rather than weight updates (Shinn et al., 2023). Self-Refine has one model generate, criticise and refine its own output (Madaan et al., 2023), representing the intrinsic case. Huang et al. (2024) find that intrinsic self-correction often fails and sometimes degrades the answer, and CRITIC emphasises that useful feedback must originate outside the model (Gou et al., 2024). Whether the two stages of the adversarial extension satisfy this condition is addressed where they are designed (Section 5.3).

3.3 Adversarial Prompting and Code Generation

LLM-GAN transfers the adversarial arrangement of generative adversarial networks (Goodfellow et al., 2014) to prompting. Its roles exchange textual feedback instead of numerical gradients, which the authors refer to as "semantic gradients" (Wang et al., 2025). On two fake-news benchmarks, it outperforms a few-shot chain-of-thought baseline and all deep-learning baselines. Removing its self-reflection stage markedly reduces performance. The paper reports no run with an open model, and its prompted baseline is GPT-3.5-turbo. It therefore provides no evidence regarding the 3B scale used here. SALF similarly pairs a generation agent with a detection agent and refines both through agent symbolic learning. The prompts serve as weights, and loss and gradient are represented as natural-language objects (Tian, Ho, and Chen, 2025). Like LLM-GAN, it focuses on fake news and uses API-served models, GPT-4o mini and DeepSeek V3. Replacing gradients with language also appears in prompt optimisation (Pryzant et al., 2023) and in TextGrad (Yuksekgonul et al., 2025).

Adversarially generated code that defeats a detector is not a new concept. ALERT renames identifiers without changing semantics to fool a CodeBERT-based detector (Yang et al., 2022), while VulGen injects vulnerabilities into normal functions from mined patterns (Nong et al., 2023). EaTVul instead has ChatGPT compose snippets whose insertion flips a detector's verdict (Liu, Cao, et al., 2024). None of these approaches closes the loop between generation

and detection: ALERT and VulGen add their output to a detector's training set in a separate gradient-based pass and without updating the generator, while EaTVul does not feed back at all. ShellForge closes this loop in the code domain by co-training a generator and a detector (Ding, 2026). It does so, however, through gradient training, on malicious PHP webshells, and with a trained classifier as the detector rather than a prompted model.

3.4 Vulnerability Datasets

Big-Vul was built from CVE-linked fix commits. Functions modified by a fix are labelled vulnerable, while the remaining functions of the same commits are labelled non-vulnerable. Each row carries the function before and after the fix (Fan et al., 2020). This rule made the dataset scalable, but it is also the source of its problems. Fix commits routinely touch functions that were never vulnerable, and because non-vulnerable functions are mostly left unchanged, whether the two versions differ is itself informative about the label. On a hand-inspected sample of Big-Vul functions labelled vulnerable, Croft, Babar, and Kholoosi (2023) found 45.7 % of the labels inaccurate. Newer datasets respond in different ways. DiverseVul widens coverage and removes duplicates, but its labels remain noisy and models trained on it generalise poorly (Chen et al., 2023). PrimeVul tightens labelling, removes duplicates and splits chronologically, and a 7B model that scores 68 % F1 on Big-Vul reaches 3 % on it (Ding et al., 2025).

Balancing per CWE, as done in the base study, keeps a trivial majority answer from reaching high accuracy. At the same time, it departs from realistic data and brings the property of F1 noted in Section 2.7 into play. Under realistic conditions, learned detectors lose more than half of their reported performance and key on incidental artefacts such as variable names (Chakraborty, Arumugam, et al., 2024; Chakraborty, Krishna, et al., 2022). Since high scores are attainable from word counts alone, Risse, Liu, and Böhme (2025) question whether function-level binary classification is a well-posed task at all. Absolute scores on Big-Vul are therefore no claim about practical detection capability. Comparisons between approaches under identical conditions, however, remain meaningful.

3.5 Research Gap

The strands surveyed above leave three openings, one for each research question.

Replication (RQ1). The comparison motivating this work relies partly on data nobody else can obtain, on single runs and on significance tests against the baseline only. Its ranking has therefore not been checked where it can be: on public data, with repeated runs and pairwise tests.

Adversarial extension (RQ2). LLM-GAN and SALF, which turn adversarial pressure into a purely prompted procedure, were demonstrated only on fake news, and neither reports a run with a small open model (Tian, Ho, and Chen, 2025; Wang et al., 2025). The code-domain work that generates adversarial or synthetic vulnerable functions returns them to the detector through gradient training (Ding, 2026; Nong et al., 2023; Yang et al., 2022) or provides no feedback at all (Liu, Cao, et al., 2024). Whether a generator and a detector can be coupled for vulnerability detection within one run using textual feedback alone and without updating parameters on either side remains untested. Part of the gap is a premise the news domain never had to test. A generated news item is fake by construction, whereas a generated function has to be checked for the flaw it is intended to contain.

Triple-Agent (RQ3). No surveyed work places a Detector–Validator chain against an adversarial generator, so whether the validating stage helps or hinders inside such a loop is open. All three questions are answered on Big-Vul, under the caveats of Section 3.4.

Chapter 4

Replicated Detection Approaches

The base study compares four approaches, but its runs did not share a setting. According to the authors, the baseline and the Dual-Agent system ran on a hosted inference service, and the fine-tuning in a separate library (Appendix B), while the released retrieval code runs locally with its own decoding. A difference between two of its approaches can therefore stem from the approach or from its setting. This replication holds everything except the approach constant. The same splits, model, sampling parameters, rule for reading a label from the answer, and evaluation are used. The chapter states the protocol, describes what each approach adds to this shared setting, and closes with an audit of what the base study leaves unspecified.

4.1 Replication Protocol

Kind of study. According to Liu, Gao, et al. (2022), this is a replication and not a reproduction, because the four approaches were re-implemented in this study's own code and run under a setup that differs from the original in the data, hardware and software stack used. When the reconstruction began, the repository linked from the base study contained only a README file. After contact with the authors, a partial release followed. It comprised a notebook with the baseline prompts and the fine-tuning run, the prompt pair of the Dual-Agent system, and the retrieval code of the RAG approach, but not the data.

Sources. The paper is the reference for what the base study did. The release and the authors' answers (Appendix B) came months after publication, and neither shows which state produced the reported numbers. They are therefore consulted only where the paper is silent. Where the sources contradict each other, the paper takes precedence, while the release takes precedence over

an answer. Three aspects are excluded from this order. First, the shared stack employed throughout this chapter, which includes the inference library and precision, sampling parameters and token limit, training library, and the evaluation state of the adapter, is established once and consistently applied to all approaches, even where a source specifies a different value for one of these components. The sole exception is the token limit for the fine-tuned approach, which retains the value specified in the notebook (Section 6.3). Second, no self-authored prompt is included in any official run. Each prompt is either Figure 1 from the paper or text from the release. The extension, whose source does not publish prompts, is exempt from this rule (Section 5.1). Third, this work defines the rule for extracting a label from the model's answer (Section 6.4). The rule of the released retrieval code matches only the two answer words, but under the Figure 1 prompt, the model often provides its verdict without using either word, so that rule would leave a large share of the test split unscored. The prescribed order is disregarded in one instance: the fine-tuning serialisation follows the notebook, contrary to the paper's claim that a single prompt was used for all techniques (Section 4.4).

Deviations. The replication differs from what the paper states in four places. First, the data consist of the public portion alone, curated here. Second, the RAG knowledge base is built from the official catalogue release (Section 4.3). Third, the fine-tuning instruction is that of the released notebook. Fourth, the hardware is another GPU. The first two and the fourth are unavoidable, the third is chosen. Every other choice concerns a value the paper leaves open and is stated where the parameter is described. Two additions address the validity threats that the original leaves open. The splits are grouped by commit, and every approach is run under three random seeds.

Success criterion. The base study makes two claims that survive the loss of its data: first the ordering RAG > SFT > Dual-Agent > Baseline, and second that each of the three interventions outperforms the baseline. The replication is judged against these two claims only. The paper's absolute values are shown for reference and enter no verdict. (a) The ranking is considered reproduced if the four approaches are ordered exactly as in the paper on F1 averaged over the five categories on the test split, and neighbouring ranks differ significantly in the prediction-level McNemar test under the seed rule of Section 6.5. McNemar's test, which compares the number of errors (see Section 2.7), contextualises the F1 order without directly assessing the metric. Any pair whose orders by F1 and by correctness disagree is reported as such. The criterion has failed when the order differs from the paper's. If the order matches but a neighbouring pair is not significant, the outcome is *reproduced*

but not resolved. (b) The baseline advantage is considered reproduced when each of the three interventions outperforms the baseline significantly in that same test. The paper's paired t-test over the five category scores is reported alongside but does not determine this result: its five observations are per-category F1 scores, not replicates, and its outcome does not show how many individual verdicts separate two approaches. The criterion was fixed in writing after the official runs of the first seed had been compared and before the runs of the further seeds existed. It is not a pre-registration, but it restates the two headline claims of the base study rather than selecting among its numbers.

4.2 Baseline (Direct Prompting)

The baseline prompts the base model directly, without retrieval, adaptation, or a second agent, and is the reference for the other three approaches (Saju, Muhtadi, and Azim, 2026). Its prompt is the classification prompt shown in Figure 1 of the paper. The paper states that this prompt was refined by trial and error and kept unchanged across all techniques, but does not specify the model variant, decoding settings, or the rule used to convert the answer into a label.

The replication uses the instruct variant, as confirmed by the authors and implied by the instruction-style prompt (Section 2.2). The prompt text is taken verbatim from the Detector prompt in the released Dual-Agent code, whose wording matches Figure 1. It is copied as a string rather than transcribed from the figure, because the figure's line breaks stem from its two-column typesetting, and even formatting changes that preserve the wording can shift a model's answers (Sclar et al., 2024). One setting from the released notebook is not retained. The limit of 256 new tokens is increased to 512 because, with the Figure 1 prompt, the verdict appears at the end and a truncated answer would not contain a label.

4.3 Retrieval-Augmented Generation

The RAG approach bases classification on the MITRE CWE catalogue, which the paper collects as PDF documents, cleans, chunks, embeds, and stores in a vector index (Chroma). At query time, the most similar chunks are retrieved and re-ranked. The paper specifies the chunk size and overlap as well as the number of retrieved candidates. However, it does not name the embedding model or re-ranker, specify how many chunks remain after re-ranking, or explain where the context is inserted into the prompt. Where the paper provides no details, the released retrieval code is used as the source. The embedding

model, re-ranker, final set size and form of the retrieval query are taken from it. The retrieved chunks are inserted into the authors' retrieval prompt, which is the Figure 1 prompt with a block of retrieved knowledge placed between the analysis steps and the instance data, in the middle position of Section 2.3.

One aspect necessarily differs from the paper: the official XML release of the catalogue is used instead of the authors' unpublished PDF collection, whose content and cleaning process cannot be recovered. The knowledge base contains the entire catalogue except deprecated entries. Category entries are indexed alongside weaknesses because two of the five target identifiers, CWE-264 and CWE-399, exist only as categories (Section 6.1). The released code is also not followed on two points in order to maintain the shared stack. It decodes with settings of its own, whereas the RAG approach here decodes like the other approaches, since a difference that stems from decoding would say nothing about the approaches. This decision was made after the first test run with the authors' settings was known. The released code also runs the model quantised to four bits at inference, while the replication runs it unquantised like every other approach. The effect of this difference was not measured.

4.4 Supervised Fine-Tuning

The fine-tuned approach trains QLoRA adapters on the base model using training examples in a three-part format consisting of instruction, input and output. The paper reports eight hyperparameters in its Table I and describes the quantisation only as four-bit or eight-bit. It does not specify a training library, batch size, sequence length, target modules or serialisation. The adapter is trained with the supervised fine-tuning trainer of TRL (von Werra et al., 2026) together with `peft` and `bitsandbytes` on a four-bit NormalFloat backbone. The eight values from Table I are used verbatim, while details omitted from the table are taken from the released notebook. For evaluation, the adapter is merged into the unquantised model shared by all approaches.

One aspect differs from what the paper states. The paper says that one prompt was used for all techniques, but the notebook trains with an instruction that differs from Figure 1. The replication follows the notebook, because the serialisation is part of the training process, and the values in Table I were obtained using it. This serialisation includes the CWE identifier but no CVE identifier and is passed to the model as plain text without the chat template, both during training and inference. The fine-tuned approach is therefore the only approach that never sees the CVE field. On two further points, the notebook is not followed. It uses Unsloth, whereas the replication uses TRL because introducing a second training framework would split the

shared software stack. Both apply the same low-rank update to the same quantised backbone. The notebook evaluates the adapter on the quantised backbone against which it was trained, whereas the replication merges it into the unquantised model so that all approaches are evaluated on the same stack. The evaluated weights therefore differ from those used to optimise the adapter (Section 2.4). Among the choices in this approach, this is the one most likely to affect the numbers, and an evaluation in the quantised state was not carried out. Finally, with the notebook's batch configuration, the step count reported in Table I covers only a small fraction of one pass over the training split. The replication retains the step count stated in the paper, and a run covering a full epoch was not carried out.

4.5 Dual-Agent System

The Dual-Agent system runs two roles of the base model in sequence: a Detector receives the classification prompt and returns a verdict with its reasoning, and a Validator reviews that output for logical inconsistencies, unsupported assumptions and errors of reasoning, and revises it. The paper presents the arrangement as a low-resource alternative to retrieval and fine-tuning and shows, in its Figure 6, a case in which the Validator overturns the Detector's verdict. The Validator's prompt is not printed.

The Detector prompt is the baseline prompt of Section 4.2. The Validator prompt is taken from the released Dual-Agent code and contains all instance fields together with the Detector's full response, so the Validator sees the original data and not only the Detector's argument. Both roles run on one loaded model under the shared decoding. No source fixes the decision rule. The paper gives none, and its Figure 6 suggests that the Validator's verdict is final, which the replication adopts. An unreadable Validator verdict stays unresolved rather than falling back to the Detector's. The number of rounds is one, as the paper's diagram implies. The Detector uses the baseline's prompt and decoding settings, so its pass constitutes a baseline run. The only difference between the two approaches is therefore the addition of the Validator stage, making the Dual-Agent system an isolated measurement of the second stage.

4.6 Reproducibility Audit

This section summarises what the published record of the base study, comprising the paper, the released artefacts and the authors' answers, determines and what it leaves open. It describes the record, not the original runs: a parameter the paper does not state may still have been fixed in the authors' setup.

Every parameter relevant to a reported result was recorded with the source that determines it and classified as specified, recovered, open, unavailable or contradictory, the last meaning that different sources determine it differently. The inventory covers data, prompt, inference, labels, RAG, SFT, Dual-Agent and evaluation and is kept in the notes of the repository (Section 6.2).

The release and the answers closed mostly configuration values. For RAG and SFT, nearly every parameter the paper leaves open was recovered. Four steps cannot be repeated by any third party: the proprietary portion of the data, present in both the training and the test set; the curation that produced the original instance sets, unknown even for their public portion; the document collection behind the RAG knowledge base; and the expert review of the labels that the paper reports. Three parameters are determined differently by different sources. The rule that turns an answer into a label matches the two answer words in the authors' answer and the released retrieval code, whereas the notebook counts an answer with neither word as not vulnerable (Section 4.1). The fine-tuning instruction differs between the paper and the notebook (Section 4.4). And the paper reports a single T4 GPU for all experiments, whereas the authors' answer places the baseline and the Dual-Agent system on a hosted inference service. The decision rule of the Dual-Agent system is fixed by no source and is chosen here (Section 4.5). Where the paper takes a position, the order of Section 4.1 settles what the replication does. What the original runs used remains unknown.

Chapter 5

Adversarial Extension

5.1 From LLM-GAN to Code

The extension transfers the two stages of LLM-GAN (Section 2.6) to vulnerability detection. The roles, the two strategy memories, the rule that exactly one side updates per round and the order of the two stages are taken from the source (Wang et al., 2025). Everything below that level had to be decided here, because the source publishes no prompt text for any role and leaves open how the memories are updated and kept within a context window. No parameter changes at any point. What the loop learns is two short lists of sentences.

The mapping. In the adversarial stage, a source item is a training record labelled *Not Vulnerable*, a clean function with its commit metadata. The Generator modifies the function to introduce a flaw corresponding to the record's weakness class and generates a new commit message. The result has the fields of a real record. The generated function serves as the vulnerable version, the clean original as the fix, while CVE, CWE, project and language are copied over. None of these fields distinguishes a generated case from a real one, so the Detector's task inside the loop is the same as its evaluation task. The self-reflection stage runs on real training records of both labels. What has no counterpart in the source is the label of a generated item. A generated news item is fake by construction, whereas a generated function is not vulnerable merely because it was generated.

Self-play, not an attacker model. Generator and Detector are prompts applied to the same model and consequently share their blind spots. In contrast, the adversarial code literature considers more stringent scenarios, such as an attacker lacking access to the target model in EaTVul (Liu, Cao, et al., 2024) or the use of semantics-preserving perturbations in ALERT (Yang et al.,

2022). Neither applies here, as the Generator intentionally alters semantics and is not blind to the Detector. The setup therefore tests whether a Detector improves through strategies derived from its own failures against a Generator of the same type, not whether an external attacker can circumvent detection.

The intended generated case. The Generator is asked for a case that meets four conditions. Under condition (i), the function contains an instance of the weakness class named in the record's metadata. Under condition (ii), it keeps the name, signature and purpose of the source function. Under condition (iii), no comment, identifier or marker points at the flaw. Under condition (iv), the commit message presents the change as routine work, because the Detector's prompt directs it to the commit message for security-related terms. Conditions (ii) to (iv) are enforced by the Generator's instructions and in part by a structural gate. Condition (i), the label assumption of the whole extension, is enforced by nothing and is checked on a sample (Section 5.5).

Prompts. Since the source publishes none, the role texts of the extension are written for this thesis and given in Appendix A. The Figure 1 prompt is not among them and reaches the Detector unchanged, so that the Detector differs from the baseline in its strategy block and in nothing else. The source's integrated prediction-and-explanation format is therefore not rebuilt, and the Reflector is prompted zero-shot, because the source's few-shot demonstrations are unpublished.

5.2 The Generator

The Generator works from training records labelled *Not Vulnerable*, drawn in a seeded order that cycles through the five weakness classes so that no class dominates the strategies the loop distils. Records reserved for the self-reflection stage are excluded. The generating prompt gives the model the project, the language, the weakness class, the clean function and, once non-empty, its strategy memory S_G, and states the conditions of Section 5.1 as requirements. The reply has three marked sections, namely the commit message, the modified function, and notes on what changed, why it is unsafe, and what makes it hard to notice. These notes are the evidence the Detector learns from when it is fooled. The sections are delimited by marker lines rather than JSON, because C code is full of braces, quotes, and backslashes, which JSON would require the model to escape.

When the Detector answers *Vulnerable*, the Generator sees the original and modified function, its notes, the Detector's assessment, and its current

memory, and adds new strategies. These are single imperative sentences that must generalise to other functions and must not name the function or the project. The Detector's update uses the same template (Section 5.3). Inside the loop, the Generator is the one role that decodes with sampling, since a deterministic Generator would return near-identical rewrites for similar inputs. As a result, each seed yields its own sequence of cases and memories, and the loop runs once per seed.

5.3 Detector, Reflector and Strategy Memories

The Detector. The Detector is the baseline of Section 4.2 with one addition. Its strategy memory S_D is placed before the unchanged Figure 1 prompt, as a numbered list under a short header that describes the entries as hints from earlier cases, not as verdicts. With an empty memory the block is omitted entirely, and the prompt is byte-identical to the baseline's. The snapshot before the first round is therefore the baseline itself, and when evaluated under the shared decoding and the same seed, it must reproduce the official baseline run verdict for verdict. This checks the pipeline, and it makes the round-zero snapshot of each seed the paired control for the gain of the loop. When the Detector was wrong, it adds strategies from one of two kinds of evidence, either the Generator's notes in the adversarial stage or the Reflector's feedback in the self-reflection stage. It is never asked to revise its verdict. Inside the loop the Detector decodes deterministically, because its verdict decides which side updates. When a snapshot is evaluated, it decodes like the replicated approaches, so that a difference from them is not confounded with the decoding.

The Reflector. In the self-reflection stage the Detector judges real training records of both labels, and every mistake is explained by the Reflector, which sees the case, the Detector's assessment and the correct label, and returns where the analysis went wrong, what in the data supports the correct answer, and one general lesson. This is externally grounded reflection in the sense of Section 3.2, the condition under which verbal self-improvement has been reported to work (Shinn et al., 2023). The adversarial stage has no such ground, since what the Detector learns there rests on the Generator's claim that its case is vulnerable. Knowing the label is also the Reflector's main risk. The label describes the function before the fix, while the prompt shows both versions, so a Detector that recognises the fix may answer *Not Vulnerable* because it reads the label as a statement about the fixed code. In explaining such a mistake, the Reflector does not require a security argument, so the resulting lesson may emphasise the labelling convention rather than detection.

The design cannot prevent this, but it keeps it visible. Every entry is one sentence, and the final memories of seed 42 are printed in Appendix A.2.

What a memory is. A strategy memory is an ordered list of sentences owned by the loop's code, not by the model. The update prompts ask only for new entries, at most two per update and at most 30 words each. The code appends them after removing duplicates. An appended memory can be compared across snapshots, whereas one rewritten by the model in every round could lose entries without trace. When a memory exceeds its limit of twelve entries, one further model call condenses it by merging entries that say the same thing. Every update instruction requires entries to generalise and forbids naming functions, identifiers or projects, so that the memory cannot carry training instances into the evaluation prompt. This is checked on the final memories.

5.4 The Adversarial Loop

Schedule. A run has the two stages of the source. In the adversarial stage each round consumes one source record. The Generator produces a case, the structural gate of Section 5.5 admits or rejects it, the Detector judges an admitted case, and exactly one side updates its memory, the Generator if the verdict was *Vulnerable* and the Detector otherwise. A rejected generation consumes a source record but not a round. In the self-reflection stage the Detector judges a fixed pool of real training records of both labels, and for every mistake the Reflector explains and the Detector updates. The second stage has a further task here. Every generated case carries the label *Vulnerable*, so the only error the adversarial stage can observe is a missed vulnerability, and S_D can accumulate nothing but reasons to answer *Vulnerable*. Only the self-reflection stage confronts the Detector with false alarms.

Budget and snapshots. There is no convergence criterion. The adversarial stage runs 200 admitted rounds and the self-reflection stage 200 records, and at the milestone rounds 0, 10, 50 and 200 and after self-reflection both memories are saved as a snapshot. The snapshot before the first round is the baseline (Section 5.3). The last one of the adversarial stage is the source's variant without self-reflection, and the one after the self-reflection stage, *final*, is the Detector the extension reports. At every milestone the current S_D is also run over a fixed, label-balanced subset of 100 validation instances, and its share of *Vulnerable* verdicts is stored as a probe of the drift towards *Vulnerable* that the adversarial stage is expected to induce.

What is measured. Detection performance, RQ2(a), is the final snapshot on the test split, compared with the baseline and the Dual-Agent system under the metrics and tests of Chapter 6. The round curve applies the same evaluation to the milestone snapshots and shows the effect of self-reflection as the difference between the last adversarial snapshot and the final one. The in-loop pass-through rate, the share of admitted cases the Detector judged *Not Vulnerable*, traces the game as the loop saw it. It is not a robustness measure, since it is computed on training material, against an adapting Generator and on labels that rest on condition (i). Robustness, RQ2(b), was designed as a flag rate on a separate hold-out of generator output paired with its clean sources. It was neither implemented nor run, because the label verification of Section 5.5 left a flag on generator output without an interpretable meaning.

5.5 Validity Checks

Two things the loop cannot establish by itself are checked separately, namely whether generated functions carry the label the loop assumes and whether the setup worked as designed.

Structural gate. Every generation passes a gate before it becomes a round. The gate rejects an output that lacks a marked section or the function, is fenced in markdown, has unbalanced braces, which also catches a cut-off function, has lost the function's name, is identical to the source, or is shorter than half or longer than two and a half times the source. A rejection is logged and never reaches either memory. Two stronger gates were rejected. Compilation would filter for compilability rather than vulnerability, since Big-Vul functions are isolated bodies without headers, type definitions or callees, and letting the same model judge its own injection would add a second uncontrolled source of error. The gate therefore checks form, not the flaw.

Hand check of generated cases. Condition (i) of Section 5.1 is measured rather than assumed. That synthetic injection needs a validation of its own is known from VulGen, whose injections do not always hold (Nong et al., 2023). After the loop run of seed 42, a seeded random sample of admitted cases is drawn, and each case is rated from its diff, its commit message, the Generator's notes and the Detector's verdict into one of four categories. A case is rated *intended* if the function plausibly contains an instance of the assigned weakness class, *other* if it contains a weakness of a different class, *broken* if it is defective without anything exploitable, and *none* if it is equivalent to the source or differs only cosmetically. The share rated *intended* is the validity

figure of the extension and is reported with an exact confidence interval. An LLM-based tooling assistant produced a preliminary rating of every case, which the author confirmed or corrected one case at a time. The check is not blind and has a single rater.

Setup checks. RQ3 builds on the RQ2 setup and presupposes that it worked as designed. Four checks establish this and are reported beside the results. Check (a) requires the loop to complete both stages without abort. Check (b) requires that the assembled prompts of all roles stay within the context window. Check (c) requires that the round-zero snapshot reproduces the baseline run of the same seed. Check (d) requires that the learned memory changes the Detector's verdicts more than a meaningless change to the prompt does, namely a different date in the chat template (Section 6.2), with both shares measured on the same validation instances under the loop's deterministic decoding. The share of *intended* cases is not among the checks. RQ2 and RQ3 learn from cases of the same Generator under the same conditions, so it bears on the validity of both questions alike.

5.6 Triple-Agent Extension

RQ3 examines whether the detecting side benefits from being the two-stage system of Section 4.5 instead of a single Detector. The Triple-Agent setup does not introduce a second loop. The loop addresses its detecting side through one interface, a party that judges a case given S_D, and RQ2 and RQ3 differ only in which party is connected. Generator, Reflector, update prompts, memories and budgets are identical, so the comparison is an ablation of one role, just as multi-agent detectors are ablated agent by agent (Widyasari et al., 2026).

The strategy block is placed before both the Detector prompt and the authors' Validator prompt, which embeds the case and the Detector's response as in the replication. The Validator's verdict is final. With an empty memory, both stages render exactly the prompts of the replicated Dual-Agent system, so the round-zero party is that system and provides the consistency check against its replicated run. Updates and reflections receive the two-stage transcript because a mistake can originate in either stage. Their wording is unchanged, so the prompt texts of RQ2 and RQ3 differ only in two labels marking the stages of the transcript. Since the Detector's own verdicts are stored, each evaluation also provides the metrics of the first stage alone under the same S_D and the share of cases in which the Validator overrode the Detector. This does not separate two-stage inference from two-stage learning. That would require the RQ2 memory evaluated with the two-stage party, which was not run.

Chapter 6

Experimental Setup

This chapter contains the values used to run the designs described in Chapters 4 and 5, to the extent that they are necessary for understanding Chapter 7 and weighing Chapter 8. It also states the statistical protocol under which the results are judged. The rationale for a configuration value can be found in the chapter in which it was selected. The complete configuration is the repository named in Section 6.2.

6.1 Data

Corpus and instances. The only corpus is Big-Vul (Fan et al., 2020), in the cleaned function-level release of its maintainers. One instance is one function row. Its label is the native flag, *Vulnerable* for vul = 1 and *Not Vulnerable* for vul = 0. The function presented for classification is the pre-commit version, so that the label describes the function *before* the change. The post-commit version accompanies it as the "after" field of the prompt. The weakness class is the commit-level CWE ID, and the CVE identifier is the corpus value, which every kept row carries, as did the base study's prompts (Appendix B, question 5). A row is kept only if its CWE ID equals one of the five target categories exactly, CWE-119, CWE-20, CWE-399, CWE-264 and CWE-200, and its language is C or C++. Rows with an empty pre-commit function are removed, as are repeated occurrences of a function body within the same commit, whereas a body that recurs across commits is kept. Two of the five identifiers are categories rather than weaknesses, and none of the five is approved for mapping real-world vulnerabilities in the current catalogue (MITRE, 2026).

Splits. A commit belongs to exactly one split, so that no function of one fixing commit appears on both sides of a split. Commits are assigned in a seeded permutation so that the five CWEs stay proportionally represented at

a ratio of 80/10/10 for train, validation and test. Within every split and every CWE the majority class is then downsampled to equal counts, and the splits are capped at 5 000, 500 and 1 000 instances, the sizes of the base study for train and test and a tenth of the training size for the validation split, which the base study does not have. The assignment was fixed before the first official run and is reused unchanged by every reported run. Across the split boundaries, ten pairs of instances share a function body up to whitespace, eight of them with opposite labels. Near-duplicates beyond whitespace are not detected. The residual risk bears on the fine-tuned configuration and the loop of the extension, the approaches that learn from the training split. Grouping by commit follows the recommendation for this corpus family (Ding et al., 2025). The deduplication that is also recommended (Croft, Babar, and Kholoosi, 2023; Ding et al., 2025) is applied within commits only. Table 6.1 gives the outcome: 5 000, 500 and 986 instances from 1 152, 131 and 200 commits, every split balanced within every CWE. The composition differs between the splits, CWE-119 making up over half of the validation split but a third of the test split.

Table 6.1: Realised splits of the Big-Vul subset: Vulnerable/Not Vulnerable pairs per CWE and split. Every split is balanced within every CWE.

CWE	train	val	test
CWE-119	1 050	139	170
CWE-20	606	44	66
CWE-399	368	21	103
CWE-264	256	23	50
CWE-200	220	23	104
Instances	5 000	500	986
Commits	1 152	131	200

The role of each split. The training split is used for fine-tuning and for both stages of the loop. The validation split serves as the researcher-in-the-loop split and is used for smoke runs, every adjustment of prompts, parser and configuration, every control run and the probe of the loop. The test split is read by the official runs alone, once per approach and seed, after the configuration of the approach was frozen against its validation results.

6.2 Model, Environment and Reproducibility

Model and hardware. All six approaches run one model, Llama-3.2-3B-Instruct (Grattafiori et al., 2024; Meta, 2024), loaded from the Hugging Face hub at a pinned snapshot through the transformers library, in 16-bit floating point (fp16) and without quantisation for every inference run. The 4-bit quantisation of QLoRA applies to the training of the fine-tuned configuration only. A prompt is passed as a single user turn through the model's chat template, whose system header carries a date pinned to `26 Jul 2024`, the default date of the template. The fine-tuned approach is the exception and receives the notebook's plain-text serialisation without the template (Section 4.4). Every reported run, including the fine-tuning and the loops of the extension, ran on one NVIDIA TITAN RTX with 24 GB of memory. The base study reports a T4 GPU and, by the authors' answer, a hosted service for two approaches (Section 4.6).

What remains non-deterministic. Model snapshot, embedder, re-ranker, catalogue version, dependencies and split reference are pinned by revision, version or hash. Because the official runs sample, a rerun reproduces a run token for token only under the same seed, batch size, attention implementation and hardware. The vector search adds a second source: it returned a different top-five list for 3 of 500 validation queries between two runs on the same index.

Repository. The repository is submitted with this thesis on a data medium. The code is one Python package with one module per approach, a shared data pipeline and one evaluation module that parses, scores and aggregates the output of every approach. Each official run is described by one configuration file. Committed with the code are the split reference, the metrics and run manifests of every official run, the experiment log and the figures. The raw corpus, the built splits, the retrieval index, the trained adapters and the raw predictions are rebuilt from that state by the commands the README documents. Providing code, configurations and prompts alongside the results follows the standard that Liu, Gao, et al. (2022) find largely unmet in software engineering and that Baltes et al. (2026) require for studies involving language models.

6.3 Per-Approach Configuration

The configuration values of every approach are recorded in the repository (Section 6.2). The rationale behind them is stated in Chapters 4 and 5. The configuration of every approach was fixed against its validation results, and the

reported test runs are the runs of that configuration. The further seeds run it with the seed as the only difference. One exception, the decoding of the RAG approach, is stated in Section 4.3. The fine-tuned approach keeps the notebook's limit of 256 new tokens. The baseline's computation recurs in the Detector pass of the Dual-Agent system and in the round-zero snapshot of the extension, both of which render the same prompt under the same decoding.

Controls on the validation split. Each control is a single run of seed 42 against a reference run that differs from it in one respect. The one that enters a reported check is the date control: a baseline run with the chat-template date `04 Sep 2026` in place of the pinned one, the one change to the model input that carries no information about the task, whose share of changed verdicts is the floor against which the effect of the strategy memory is read (setup check (d) of Section 5.5, under the loop's deterministic decoding). Further controls, the stricter prompt variant of the authors' notebook, RAG with random or without retrieved chunks, and a greedy Validator, were run on the validation split. They decide nothing and are reported in the repository.

6.4 Evaluation Metrics

The six approaches differ in how the raw model output is produced and in nothing that follows: one parser, one scoring module and one aggregation serve all of them.

From text to label. The label is read from the raw output by one parser. It searches the conclusion of the output first and widens to the last 600 characters and then to the full text. Within a window an explicit answer word decides first, an explicit refusal ends the search with no label, and otherwise a verdict phrase such as "fixes a vulnerability" or "no security issue" is read through the question of Figure 1. An output in which no window yields a verdict has no label. The rule is constructed here, since the paper documents none and the released rule would leave a large share of the outputs unscored (Section 4.1). One output form of the fine-tuned approach, `Classification: Vulnerable or Not Vulnerable`, contains both answer words, and the first-match rule reads it as *Vulnerable*. How often it occurs and what the two other possible readings would change is reported in Section 7.2.2. Two policies score an output without a label. Under `drop`, the rule the authors state for their own runs (Appendix B, question 3), the instance is excluded from the confusion counts. Under `wrong`, it is counted as a misclassification. The `drop` figures are primary, and every comparison is repeated under `wrong`.

Metrics. *Vulnerable* is the positive class (Appendix B, question 7). Per CWE, the confusion counts yield accuracy, precision, recall and F1. The overall row, "All (Avg)" in the base study, is the unweighted mean of the five per-CWE values, which is also what keeps the two studies comparable, since the pooled figures would weight each category by its share of the split. F1 averaged over the five CWEs is the metric on which the ranking of RQ1 is defined. The Matthews correlation coefficient and the predicted-positive rate (Section 2.7) are reported beside it as macro values. An MCC that is undefined, as for a classifier that always gives the same answer, is set to 0 by convention. Every metric is reported as the mean and the sample standard deviation over the three seeds, descriptively. A snapshot of the loop is evaluated exactly as the baseline is, and the share of the hand check rated *intended* is reported with an exact binomial (Clopper-Pearson) 95 % confidence interval.

6.5 Statistical Analysis

Every test is two-sided at $\alpha = 0.05$. Two approaches are compared with the exact McNemar test (Fagerland, Lydersen, and Laake, 2013; McNemar, 1947) on their paired correct/incorrect outcomes: the instances of the test split are matched by identifier, and the p-value is the two-sided exact binomial probability of the observed split of the discordant pairs. Under the primary policy `drop` only pairs in which both approaches produced a readable label enter. The test is on the level of individual verdicts and tests correctness, whereas the ranking of RQ1(a) is defined on F1. Pairs whose order by F1 and by accuracy disagree are marked in Section 7.2.1.

Seeds. Every approach is run on the test split under the seeds 42, 43 and 44, and comparisons pair runs of the same seed. What a seed varies differs between approaches: for the baseline, the RAG approach and the Dual-Agent system it is the sampling of the decoder alone. For the fine-tuned configuration it is in addition the initialisation of the adapter. For the extension it is in addition the Generator's sampling and with it the whole course of the loop. The standard deviations over seeds are therefore not comparable across approaches, and no inference rests on them. The ranking of RQ1(a) is read from the macro-F1 averaged over the three seeds, with the ranking within each seed reported beside it. The significance of one comparison is decided by the *seed rule*: McNemar is computed for each of the three seed pairs, the p-value of the comparison is the largest of the three, $p_{\max} = \max(p_{42}, p_{43}, p_{44})$, the intersection-union test of the three per-seed hypotheses (Berger, 1982), and the comparison is significant when $p_{\max} < \alpha$ and the discordant counts favour

the same side in all three seeds. The extension to three seeds can only remove a significance that the seed-42 comparison would have shown, never add one.

Families and correction. The correction for multiple comparisons follows the logical form of each claim. RQ1(a) claims an order of four approaches, which holds only if all three neighbouring pairs are significant. That is a conjunction, which holds the level α without correction, and the three non-neighbouring pairs are reported descriptively. RQ1(b) claims that each of the three interventions outperforms the baseline. Read as one conjunction it needs no correction either, and because the three pairs are also read as findings in their own right, Holm-adjusted p-values over the three are reported beside the raw ones. RQ2(a) rests on two comparisons, the final GAN Detector against the baseline and against the Dual-Agent system, whose two $p_{\max}$ values are adjusted together by the Holm procedure (Holm, 1979). RQ3 rests on one comparison and needs no correction.

Sensitivity and secondary test. Every comparison is computed a second time under the policy `wrong`, with the same seed rule and the same families. The test of the base study, a two-sided paired t-test over the five per-CWE F1 values of two approaches, each averaged over seeds, is reported alongside but decides nothing. If the two tests disagree, that case is treated as a result about the statistical design of the base study and is not resolved silently in favour of either test.

Chapter 7

Results

This chapter reports the official runs on the test split, 986 instances, each approach under the seeds 42, 43 and 44. Unless stated otherwise, figures are macro values over the five CWEs under the primary policy `drop`, given as the mean and standard deviation over the three seeds (Section 6.4), and significance follows the seed rule of Section 6.5. Their interpretation is the subject of Chapter 8.

7.1 Overview

Figure 7.1 and Table 7.1 hold the four metrics the base study reports, for all six approaches on the test split. Table 7.2 adds the Matthews correlation and the predicted-positive rate, which the base study does not report. The two round-zero snapshots of the extension have no rows of their own: by the setup checks of Sections 7.3.1 and 7.4 they reproduce the baseline and the Dual-Agent run of the same seed text for text.

Level. Only the fine-tuned configuration exceeds an F1 of 0.667, the value reached by a classifier that answers *Vulnerable* on every instance of a balanced split (Section 2.7). Every replicated F1 lies below the value the base study reports for the same approach. Those reference values are shown for orientation and enter no verdict (Section 4.1).

Order. By mean macro-F1 the six approaches order SFT, the final GAN Detector, RAG, the baseline, the final Triple-Agent party and the Dual-Agent system. Only the lead of the fine-tuned configuration is wide: it is 0.133 ahead of the second, while the five below it are separated by 0.018 to 0.032, less than the 0.020 to 0.052 by which the three seeds of a single approach spread.

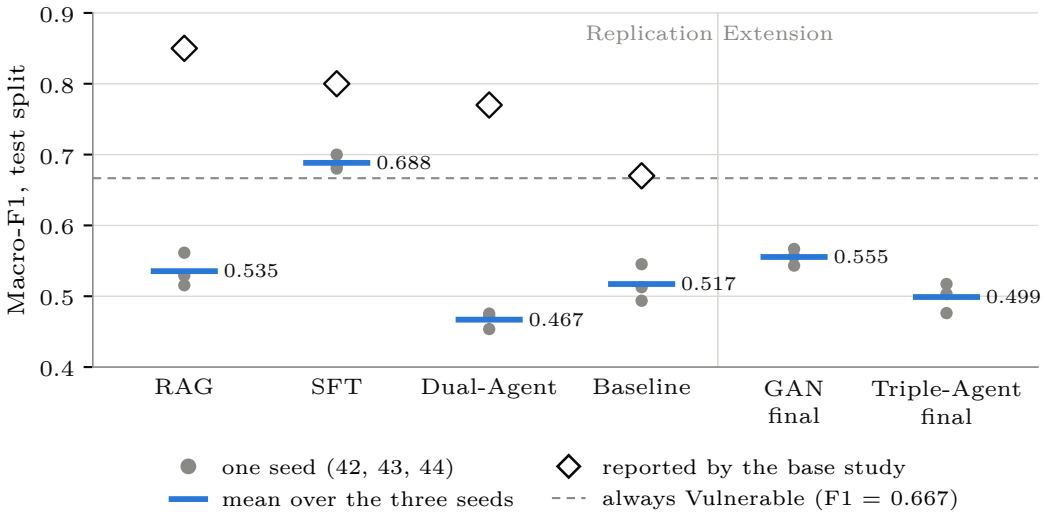


Figure 7.1: Macro-F1 on the test split per seed, with the mean of each approach and the value the base study reports. Dashed line: the F1 of always answering *Vulnerable*.

The order of those five is an orientation and not a result. Which of their differences survives a test is decided in the sections that follow. Between the extremes the six approaches differ about three times as much in recall, 0.340, as in precision, 0.111: the fine-tuned configuration labels 0.604 of all instances *Vulnerable*, the five prompting approaches between 0.380 and 0.417, and their Matthews correlations lie between 0.044 for the Dual-Agent system and 0.202 for the final GAN Detector.

7.2 Replication Results (RQ1)

RQ1 asks whether the order the base study reports, RAG > SFT > Dual-Agent > Baseline, holds in the replication (a), and whether each of the three interventions outperforms the baseline significantly (b). Neither part is reproduced: the observed order differs from the paper's in two places, and of the three interventions only the fine-tuned configuration outperforms the baseline significantly, a result that does not survive the Holm adjustment.

7.2.1 Observed Ranking vs. the Base Study

By mean macro-F1 the replication orders the approaches SFT > RAG > Baseline > Dual-Agent, and each of the three seeds yields the same order. Com-

Table 7.1: The six approaches on the test split in the four metrics of the base study, mean and standard deviation over the three seeds. The lower rows are the values the base study reports.

Approach	F1	Acc.	Prec.	Rec.
RAG	0.535 ± 0.024	0.576 ± 0.019	0.604 ± 0.025	0.491 ± 0.021
SFT	0.688 ± 0.010	0.657 ± 0.015	0.636 ± 0.022	0.761 ± 0.038
Dual-Agent	0.467 ± 0.012	0.521 ± 0.011	0.526 ± 0.014	0.421 ± 0.010
Baseline	0.517 ± 0.026	0.576 ± 0.020	0.617 ± 0.025	0.455 ± 0.026
GAN final	0.555 ± 0.012	0.595 ± 0.008	0.632 ± 0.014	0.512 ± 0.021
Triple-Agent final	0.499 ± 0.021	0.549 ± 0.014	0.563 ± 0.015	0.451 ± 0.024
RAG (paper)	0.85	0.86	0.88	0.84
SFT (paper)	0.80	0.81	0.85	0.76
Dual-Agent (paper)	0.77	0.78	0.80	0.76
Baseline (paper)	0.67	0.65	0.68	0.66

pared with the paper's order, the replication exchanges the first two and the last two positions. By accuracy the order is the same, except that in seed 44 the baseline is ahead of the RAG approach by 0.006, so the pair RAG vs. Baseline orders differently by F1 and by accuracy in that seed.

Table 7.2: Matthews correlation (MCC) and predicted-positive rate (PPR) on the test split, mean and standard deviation over the three seeds.

Approach	MCC	PPR
RAG	0.159 ± 0.039	0.415 ± 0.004
SFT	0.326 ± 0.026	0.604 ± 0.048
Dual-Agent	0.044 ± 0.023	0.400 ± 0.002
Baseline	0.163 ± 0.040	0.380 ± 0.008
GAN final	0.202 ± 0.018	0.417 ± 0.023
Triple-Agent final	0.101 ± 0.029	0.402 ± 0.010

Table 7.3 gives the prediction-level tests for the six pairs. Of the three pairs that are neighbours in the paper's order, RAG vs. SFT and SFT vs. Dual-Agent are significant under the seed rule, both in favour of the fine-tuned configuration, so the first is significant in the direction opposite to the paper's order. Dual-Agent vs. Baseline is not significant: the baseline is correct more often in all three seeds, but seed 43 leaves the pair within chance and sets the $p_{\max}$ of 0.193. Criterion (a) of Section 4.1 is therefore not met. Criterion (b) requires each intervention to outperform the baseline significantly: the fine-

tuned configuration does ($p_{\max} = 0.031$), the RAG approach does not ($p_{\max} = 0.957$, the discordant pairs favouring the baseline in seed 42 and the RAG approach in seeds 43 and 44), and the Dual-Agent system is the less often correct side in every seed. Read as three findings with the Holm adjustment, none of the three is significant, since the adjusted value of the fine-tuned configuration is 0.094. Under the policy `wrong` no pair changes its verdict under the seed rule, and the paired t-test of the base study is significant for exactly the same three pairs as the seed rule. Only the Holm reading moves. Under `wrong` the adjusted value of the fine-tuned configuration falls to 0.037, so its advantage over the baseline survives the adjustment under that policy, but not under the primary one.

Table 7.3: Pairwise McNemar tests on the test split, grouped by the criterion of Section 4.1 that each pair decides. p_{Holm} adjusts $p_{\max}$ within block (b). "Better" is the side correct more often in every seed, otherwise "mixed". The t-test is the base study's and decides nothing.

	McNemar p per seed						
Pair	42	43	44	$p_{\max}$	p_{Holm}	Better	t-test p
(a) Neighbours in the paper's order							
RAG vs. SFT	0.008	0.005	0.002	0.008	–	SFT	0.002
SFT vs. Dual-Agent	<0.001	<0.001	<0.001	<0.001	–	SFT	<0.001
Dual-Agent vs. Baseline	<0.001	0.193	0.004	0.193	–	Baseline	0.151
(b) Interventions against the baseline							
RAG vs. Baseline	0.957	0.827	0.742	0.957	0.957	mixed	0.404
SFT vs. Baseline	0.031	0.004	0.001	0.031	0.094	SFT	0.001
Dual-Agent vs. Baseline	<0.001	0.193	0.004	0.193	0.385	Baseline	0.151
Remaining pair							
RAG vs. Dual-Agent	<0.001	0.088	<0.001	0.088	–	RAG	0.070

7.2.2 Per-Approach Performance

The fine-tuned configuration leads the other three replicated approaches on every metric of Table 7.1 and in F1 in every CWE. Its F1 per CWE lies between 0.646 and 0.720, and the highest F1 of any other replicated approach in any CWE is 0.575. The RAG approach and the baseline are equal in accuracy to three decimals and differ in F1 by less than the seed standard deviation of either. The Dual-Agent system is last on every metric, with a Matthews correlation of 0.044. Outputs without a label occur in 1.8 % of the RAG answers and 0.8 % of the baseline answers and not at all for the other two approaches. Counted as errors under the policy `wrong`, they lower the macro-F1

of the RAG approach by 0.011 and of the baseline by 0.005, and the order is the same under both policies. The fine-tuned configuration has no output without a label, but one output form that contains both answer words, `Classification: Vulnerable or Not Vulnerable`, which the parser reads as *Vulnerable* (Section 6.4). It occurs in 22, 46 and 53 of the 986 outputs of the three seeds. Read as unreadable instead, or as *Not Vulnerable* by the rule of the authors' notebook, the mean macro-F1 of the configuration moves from 0.688 to 0.680 or 0.660. Its rank is unchanged, and its advantage over the baseline stays significant under the seed rule in either reading ($p_{\max} = 0.017$ and 0.031).

The Detector pass of the Dual-Agent system reproduces the baseline run of the same seed text for text in all three seeds, so the Dual-Agent figures are those of the baseline after the Validator's revision. The Validator changes the Detector's verdict on 552, 566 and 545 instances, out of the 977, 980 and 976 on which that verdict could be read, 0.567 of them on average, and in every seed it turns more correct verdicts into wrong ones than the reverse. No Validator verdict was unreadable, so falling back to the Detector's verdict, which Section 4.5 rejects, would yield figures identical to the reported ones.

7.3 Adversarial Extension Results (RQ2)

RQ2 asks whether the adversarial approach improves over the baseline and the Dual-Agent system in detection performance on the test split (a) and in robustness (b). The final Detector is correct significantly more often than the Dual-Agent system, but not more often than the baseline. Part (b) was not measured: the hold-out of Section 5.4 was never run, because the hand check found no generated case that carries the intended flaw.

7.3.1 Generated Functions and Setup Checks

Hand check. From the loop of seed 42, a random sample of 30 admitted cases was rated by hand (Section 5.5). Not one of them carries the weakness the Generator was asked to introduce. Table 7.4 gives the four categories and the number of cases in each. The share of *intended* cases is therefore 0, with a 95 % confidence interval reaching up to 0.116. The share of cases with any weakness, *intended* or *other*, is 0.20, with an interval from 0.077 to 0.386. The tooling assistant's proposed rating was confirmed for all 30 cases. Five of the 30 had been judged *Vulnerable* by the in-loop Detector, three of them in the category *none*.

Table 7.4: The hand check of 30 admitted cases from the GAN loop of seed 42: the four categories, the number of cases in each, and how many of them the in-loop Detector had judged *Vulnerable*.

Category	The generated function ...	Cases	Judged *Vulnerable* in the loop
intended	plausibly contains an instance of the assigned weakness class	0	0
other	contains a weakness of a different class	6	1
broken	is defective without anything exploitable	11	1
none	is equivalent to its source or differs only cosmetically	13	3
Total		30	5

The loops. Each seed ran one loop of 200 adversarial rounds followed by 200 self-reflection items on the training split. Table 7.5 gives three figures per loop run. The structural gate rejected between 22 % and 24 % of the Generator's outputs, mostly for formal defects such as a markdown fence around the function or a length outside the permitted band. Of the admitted cases, the in-loop Detector judged between 75 % and 90 % *Not Vulnerable*. This pass-through rate is computed on training material, against an adapting Generator and on cases of which none was rated *intended*, so it is not a robustness measure. In the self-reflection stage the Detector answered between 0.525 and 0.570 of the items correctly.

Table 7.5: Per loop run, for the GAN loops of RQ2 and the Triple-Agent loops of RQ3 (Section 7.4): share of Generator outputs rejected by the structural gate, share of admitted cases judged *Not Vulnerable* in the loop, and accuracy on the self-reflection items.

Loop	Seed	Rejected by the gate	Judged *Not Vulnerable*	Self-reflection accuracy
GAN (RQ2)	42	22 %	75.0 %	0.560
	43	24 %	90.0 %	0.525
	44	24 %	84.5 %	0.570
Triple-Agent (RQ3)	42	27 %	54.0 %	0.595
	43	26 %	68.0 %	0.535
	44	24 %	61.0 %	0.605

Setup checks. All four checks of Section 5.5 hold. (a) All three loops completed both stages. (b) The longest prompt of any role had 20 444 tokens, well within the context window. (c) In all three seeds the round-zero snapshot reproduced the baseline run of the same seed on the test split text for text, in all 986 verdicts and output texts. (d) On the validation split of seed 42 under the loop's deterministic decoding, the final memory changed 0.250 of the verdicts readable in both runs relative to the empty memory, and a meaningless change of the chat-template date changed 0.113, so the memory has the larger effect. Under the sampling decoding of the evaluations the two shares are 0.324 and 0.303 and thus close.

7.3.2 Detection Performance vs. Baseline and Dual-Agent

RQ2(a) compares the final snapshot of each loop, the Detector with the memory it holds after both stages, against the two approaches that likewise use prompting alone. Round zero is the baseline itself, so the comparison with the baseline shows what the loop adds to the same model and the same prompt. The final Detector reaches a macro-F1 of 0.555 ± 0.012 against 0.517 ± 0.026 for the baseline, and labels 0.417 of the instances *Vulnerable* against 0.380. Table 7.6 gives the tests. Against the Dual-Agent system the final Detector is correct more often in every seed, and the comparison is significant, also after the Holm adjustment. Against the baseline it is not: the baseline is correct more often in seed 42, the final Detector in seeds 43 and 44, and $p_{\max}$ is 0.866. Neither verdict changes under the policy `wrong`. The paired t-test of the base study disagrees with the seed rule on both pairs: it finds the final Detector ahead of the baseline ($p = 0.049$), but not ahead of the Dual-Agent system ($p = 0.053$). The seed rule decides.

Table 7.6: Tests of RQ2(a) on the test split: the final GAN Detector against the two prompting approaches, in the format of Table 7.3. p_{Holm} adjusts the two $p_{\max}$ values together.

	McNemar p per seed						
Pair	42	43	44	$p_{\max}$	p_{Holm}	Better	t-test p
GAN final vs. Baseline	0.866	0.008	0.246	0.866	0.866	mixed	0.049
GAN final vs. Dual-Agent	<0.001	<0.001	<0.001	<0.001	<0.001	GAN final	0.053

Figure 7.2 shows how the loop reaches its final snapshot. Seed 42 was evaluated at every snapshot: its F1 is unchanged after ten admitted rounds, lower at round 50 and lowest at round 200, the end of the adversarial stage,

and back at the level of round zero after self-reflection. The difference that Section 5.4 names in advance, between round 200 and the final snapshot, is not significant ($p = 0.312$). Of the ten pairwise comparisons between the snapshots of this seed, one falls below 0.05 without correction: round 10, the highest point of the curve, is correct more often than round 200, the lowest ($p = 0.045$). After a Holm adjustment over the ten comparisons, none of them is significant. Seeds 43 and 44 were evaluated only at round zero and at the end. In both, the final snapshot lies above round zero. On the validation probe, the share of *Vulnerable* verdicts rises in all three seeds from 0.17 at round zero, to 0.28, 0.23 and 0.20 at round 200 in seeds 42, 43 and 44. After self-reflection it falls back to 0.21 in seed 42, but rises further to 0.27 in seeds 43 and 44.

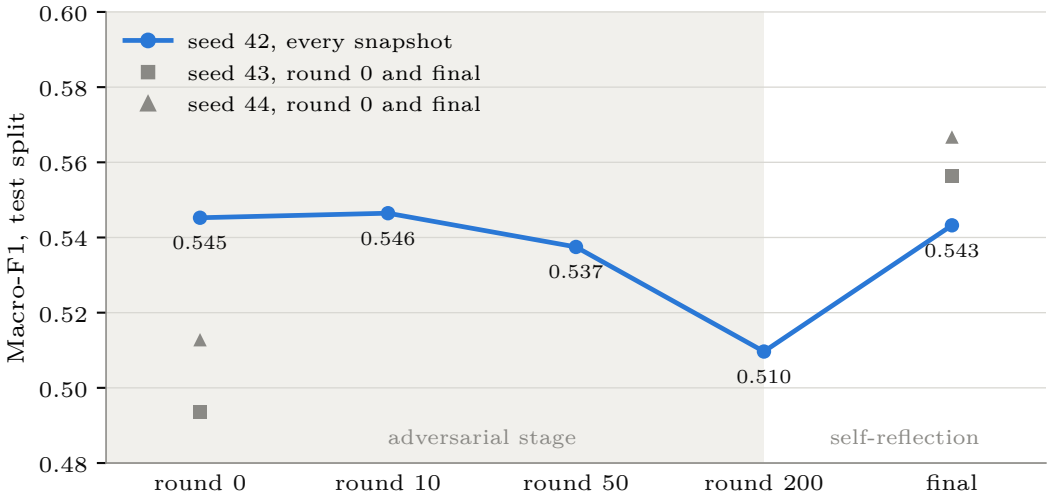


Figure 7.2: Macro-F1 on the test split along the GAN loop. Seed 42 was evaluated at every snapshot, seeds 43 and 44 at round zero and after the self-reflection stage (final). Rounds count admitted adversarial rounds. The horizontal axis is ordinal.

7.4 Triple-Agent Results (RQ3)

RQ3 asks whether the adversarial setup improves further when the full Dual-Agent system takes the place of the single Detector. It does not. The final Triple-Agent party is correct less often than the final GAN Detector in every seed, though not significantly. The Triple-Agent loops ran under the configuration and the seeds of the GAN loops, and checks (a) to (c) of Section 5.5 hold for them as well, while check (d) was run for the GAN loop only, whose memory mechanism the Triple-Agent loop shares: all three loops completed

both stages, the longest prompt had 20 919 tokens, and in all three seeds the round-zero party reproduced the Dual-Agent run of the same seed on the test split text for text. The lower block of Table 7.5 shows that the two-stage party let fewer generated cases pass as *Not Vulnerable* than the single Detector in every seed.

The final party reaches a macro-F1 of 0.499 ± 0.021, against 0.467 ± 0.012 at round zero and 0.555 ± 0.012 for the final GAN Detector. Table 7.7 gives the tests. The final GAN Detector is correct more often in every seed, but seed 44 leaves the pair within chance ($p_{\max} = 0.075$), so the comparison is not significant. Against its own round zero the final party is correct more often in every seed, but not significantly either ($p_{\max} = 0.676$). Both verdicts are the same under the policy `wrong` and under the paired t-test.

Table 7.7: Tests of RQ3 on the test split, in the format of Table 7.3: the final Triple-Agent party against the final GAN Detector, and against its own round zero, the replicated Dual-Agent system. No Holm value applies, since RQ3 rests on one comparison.

	McNemar p per seed						
Pair	42	43	44	$p_{\max}$	p_{Holm}	Better	t-test p
Triple final vs. GAN final	<0.001	0.027	0.075	0.075	–	GAN final	0.089
Triple final vs. Triple round 0	0.676	0.111	0.027	0.676	–	Triple final	0.189

Because every evaluation also stores the Detector's own verdicts, the two stages of the party can be read apart. Table 7.8 sets them beside the final GAN Detector. Under the final memory, the Detector stage alone reaches the level of the final GAN Detector on average and is ahead of it in seeds 43 and 44. The Validator then changes about half of the Detector's verdicts, somewhat fewer than at round zero, and in every seed its revision lowers the F1 by between 0.04 and 0.08. The Detector stage alone is a diagnostic, not an official comparison, and no test is reported for it.

Table 7.8: The two stages of the Triple-Agent party on the test split, per seed: macro-F1 of the party's final verdicts, of the Detector's own verdicts before the Validator, and of the final GAN Detector of the same seed, plus the share of verdicts the Validator changed, at round zero and in the final snapshot.

	Macro-F1			Verdicts changed by the Validator	
Seed	Triple final	Detector stage alone	GAN final	round 0	final
42	0.476	0.527	0.543	0.565	0.519
43	0.517	0.561	0.556	0.578	0.445
44	0.503	0.587	0.567	0.558	0.505
Mean	0.499	0.558	0.555	0.567	0.490

7.5 Cost and Efficiency

Figure 7.3 sets the output length of each approach beside its macro-F1. The fine-tuned configuration answers with about five tokens, the target string it was trained on, and has the highest F1. The five prompting approaches generate between 261 and 498 tokens per test instance and reach a lower F1. The two two-stage parties, which call the model twice, generate the most and score lowest. On the hardware of Section 6.2, a test run takes about ten minutes for the fine-tuned configuration, about two GPU hours for the baseline, the RAG approach and a GAN snapshot, and about four for the Dual-Agent system and a Triple-Agent snapshot. Before evaluation, the fine-tuned configuration needs a training run of about four minutes per seed, the GAN extension a loop of 4.5 hours and the Triple-Agent extension one of 6.5 hours, each averaged over the three seeds.

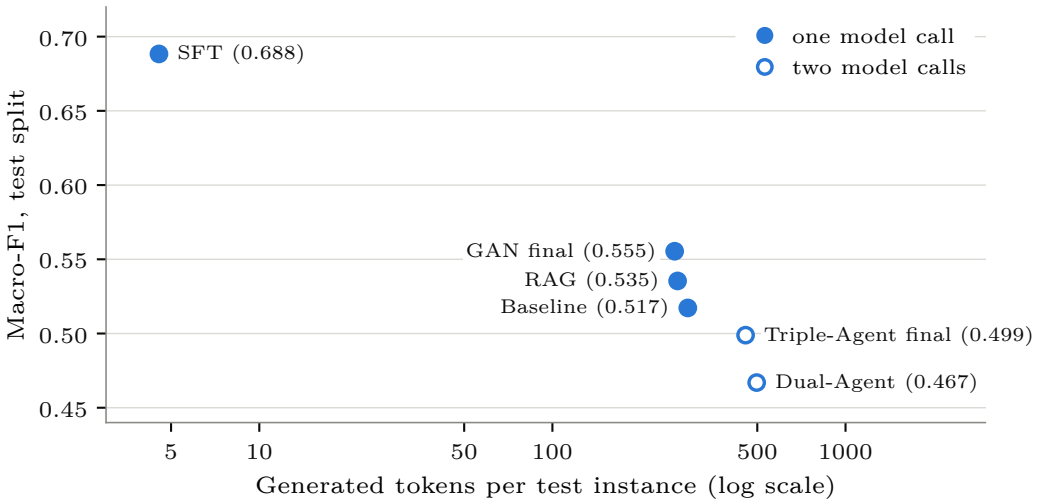


Figure 7.3: Generated tokens per test instance against macro-F1 on the test split, one point per approach. Tokens are counted with the model's tokeniser over all model calls of an approach and averaged over the three seeds. The horizontal axis is logarithmic. One-off costs are not included.

7.6 Qualitative Error Analysis

Generated cases. Two cases of the hand check illustrate the categories of Section 7.3.1. Figure 7.4 shows each with the flaw the Generator's notes claim and an excerpt of the diff from the clean source to the generated function. Case (a) claims a flaw that the change does not contain: it removes a branch-prediction hint and leaves the behaviour unchanged, so it was rated *none*. The in-loop Detector nonetheless judged it *Vulnerable*. Case (b) introduces the flaw its notes describe, replacing an early error return by a retry loop without a bound, but the flaw is of a different class than the assigned improper input validation, so it was rated *other*. The in-loop Detector judged it *Not Vulnerable*, and the case entered the Detector's update as a missed vulnerability.

The final memories. Section 5.3 anticipated that the Detector memory might state Big-Vul's labelling convention rather than ways to recognise a flaw. The final GAN memories S_D contain entries of both kinds, and a third kind besides, general advice on how to write code. Table 7.9 gives one entry of each kind verbatim, and Appendix A.2 prints the memories of seed 42 in full. The reading is qualitative. The entries were not classified one by one.

(a) Assigned CWE-264, Linux kernel. Rated *none*, judged *Vulnerable* in the loop. Generator's notes: a null pointer dereference and a use-after-free.

```
-  if (unlikely(need_fallback(sctx->key_len)))
+  if (need_fallback(sctx->key_len))
       return fallback_blk_dec(desc, dst, src, nbytes);
```

(b) Assigned CWE-20, Chrome. Rated *other*, judged *Not Vulnerable* in the loop. Generator's notes: an infinite loop that can deny service.

```
-  if (!GetBrowserFromWindowID(this, window_id, &browser, &error))
-    return RespondNow(Error(error));
+  while (!GetBrowserFromWindowID(this, window_id, &browser, &error)) {
+    params.reset();
+    params = tabs::GetSelected::Params::Create(*args_);
+    EXTENSION_FUNCTION_VALIDATE(params.get());
+    if (params->window_id.get())
+      window_id = *params->window_id;
+  }
```

Figure 7.4: Two generated cases of seed 42 with their rating in the hand check and an excerpt of the diff from the clean source. Removed lines are set in grey.

Table 7.9: Three kinds of entry in the final GAN Detector memories, each with one entry verbatim and the seed of its memory.

Kind	The entry ...	Example (seed)
Labelling convention	relates a change to a fix	"Verify code changes appear identical before and after to ensure no new vulnerabilities are introduced." (42)
Detection heuristic	names a property that can be checked on a given function	"Assess the potential for off-by-one errors in string manipulation code." (43)
Coding advice	says how code should be written	"Use meaningful variable names to avoid misleading code reviews and make vulnerabilities more apparent." (42)

Many entries of the final Generator memories S_G describe ways to obscure code, such as indirect calls, misleading names or suppressed error handling, rather than a flaw to introduce. Fewer name a flaw, and some describe defensive measures instead, such as "Prevent integer overflows: Cast values to prevent unchecked integer overflows and data corruption" in seed 44. None of the final memories of any seed or setup contains an identifier-shaped token, and a comparison with the code-token trigrams of the test functions yields only fragments of ordinary English such as "to create a", so the memories carry no training instance into the evaluation prompt.

Chapter 8

Discussion

This chapter interprets the results of Chapter 7, beginning with the answers to the research questions. One observation runs through the sections after them: on Big-Vul, and with a model of three billion parameters, the prompting approaches differ in how often they answer *Vulnerable* and, with one exception, little in how well they tell vulnerable from non-vulnerable functions. The exception is the Dual-Agent system, whose second stage removes most of that separation.

8.1 Answering the Research Questions

Every verdict below is decided by the seed rule of Section 6.5. The policy `wrong` and the paired t-test of the base study are reported in Chapter 7 as sensitivity analyses and decide nothing.

RQ1(a): not reproduced. The observed order departs from the paper's in two places, and the two departures are not equally firm. The fine-tuned configuration leads RAG significantly under every analysis, so here the replication contradicts the paper's order rather than merely failing to confirm it. The Dual-Agent system is behind the baseline on every metric and less often correct in every seed, but not significantly, because seed 43 leaves the pair within chance. Between RAG and the baseline nothing is resolved. RAG is ahead in F1 in every seed but slightly behind in mean MCC, and the discordant pairs favour neither side.

RQ1(b): not reproduced. Only the fine-tuned configuration outperforms the baseline significantly. RAG does not differ significantly from the baseline, and the Dual-Agent system lies on the other side of it, so the claim fails for

two of the three interventions. Even the one success is fragile, since under the primary policy it does not survive the Holm adjustment. On every pair of RQ1, the base study's own test reaches the same verdict as the seed rule.

RQ2(a): better than the Dual-Agent system, not better than the baseline. The final GAN Detector is correct more often than the Dual-Agent system in every seed, and the comparison remains significant after the Holm adjustment. Against the baseline the direction changes between seeds. The two comparisons do not carry equal weight. Round zero is the baseline, which is itself ahead of the Dual-Agent system in every seed, so the first comparison reflects the weakness of that system's second stage at least as much as the loop. What the loop itself adds is measured only by the comparison with the baseline, and there it shows no consistent effect. Neither comparison can show adversarial learning. The hand check found no sampled case with the intended flaw, so the Detector updates of the adversarial stage followed verdicts on largely harmless rewrites, a signal that can only push towards *Vulnerable*. Accordingly, the final Detector answers *Vulnerable* more often than round zero, and F1 on balanced data rewards that direction (Section 2.7).

RQ2(b): not measurable as posed. Section 1.3 made part (b) conditional on the validation of generated cases, and the validation removed the premise of the measurement, since none of the 30 sampled cases carries the assigned weakness, and most carry no weakness at all. A flag rate on such cases would mostly record reactions to harmless rewrites, where not flagging is often the correct answer, so the hold-out was not run. Part (b) remains unanswered, and the reason is itself a result: in this setup the approach already fails at the Generator. Only the validation step that the extension added to its template made this visible. The loop's own signals would not have shown it.

RQ3: no improvement. The final Triple-Agent party does not improve on the final GAN Detector. The GAN Detector is correct more often in every seed, and the comparison misses significance only because seed 44 leaves the pair within chance. The two-stage party improves on its own round zero in every seed, but not significantly, and it ends below the final GAN Detector. The answer is therefore negative in direction and unresolved in significance. Section 8.3 traces the gap to the second stage. Under the same memory, the Detector stage alone is ahead of the party in every seed.

8.2 Why the Order Differs

Where the decision lies. On balanced data F1 rewards answering *Vulnerable* more often (Section 2.7), and all five prompting approaches stay below the 0.667 that always giving this answer would reach. Measured by the Matthews correlation (Table 7.2), the ends of the RQ1 order stay in place, with the fine-tuned configuration first and the Dual-Agent system last and close to chance, but RAG and the baseline become indistinguishable. In the middle of the order, the F1 ranking is therefore a ranking of predicted-positive rates. The success criterion, which restates the base study's claims in the base study's metric, is unaffected. What changes is what a difference in that metric can be taken to show.

Fine-tuning. The fine-tuned configuration is the only approach that improves separation, and it also answers *Vulnerable* far more often than the others. One counter-reading has to be taken seriously: labelled data teach the task, but they also teach the convention by which the dataset assigns its labels. In Big-Vul, a function changed by a fixing commit is vulnerable and the unchanged functions of that commit are not, and the prompt shows both versions (Section 8.5.1). On the test split, whether a function is identical before and after its commit determines the label without exception, so a model that had learned this shortcut would be nearly perfect. The fine-tuned configuration judges only about half of the unchanged functions correctly, so it has not learned the shortcut as such. The results are consistent with the configuration having learned from examples a reading of the label that an instruction does not convey, and equally consistent with part of that reading being Big-Vul's convention. They do not show which part is larger. Training covers a small fraction of one epoch, which suggests, without measuring it, that the configuration has not reached its ceiling (Section 4.4).

Retrieval and the second agent. RAG adds retrieved text but no separation. Its lead over the baseline in F1 comes with a higher predicted-positive rate and without a higher Matthews correlation, so the model appears to react to the presence of domain text more than to how well it fits the instance. This is plausible for a context block in the middle of the prompt (Liu, Lin, et al., 2024). With larger models, knowledge-level retrieval has improved the separation of vulnerable from patched code (Du et al., 2025). The Dual-Agent system trails the baseline because of its second stage. Its first stage is the baseline, text for text, and the Validator then changes more than half of the verdicts, more than a different seed does, and in every seed it turns more correct verdicts into wrong ones than the reverse (Section 7.2.2). This matches

what has been reported for intrinsic self-correction without an external signal, which often fails and can make the answer worse (Huang et al., 2024). The Validator sees the same evidence as the Detector, plus the Detector's verdict, and nothing makes its judgement better than the one it replaces.

What remains comparable with the base study. The replication meets the base study at the level of its claims, not of its numbers. Every replicated F1 lies below the value the base study reports, and this gap is not a verdict on either study. The causes this study could vary without new data, the policy for unreadable answers and the seeds, move its numbers by amounts that are small compared with the gap. The causes that could explain the gap, the proprietary data in the original test set, the unpublished curation of its instances, the expert review of its labels, the serving of its runs and the prompt behind its baseline table, can be tested by no third party. The order is where the comparison has content, and there the replication contradicts the base study in one place. The fine-tuned configuration leads RAG under every analysis. It is notable that the approach the base study ranks first is the one whose distinguishing input could not be rebuilt. The documents behind its knowledge base are unpublished, and the catalogue that replaces them adds no separation for this model. Elsewhere the replication merely fails to confirm the base study, and neither study has to be in error for this pattern to arise, because an intervention whose effect depends on what it is given can help under one set of data and conditions and not under another.

What the statistical design can show. The base study's design, a single run per configuration and a paired t-test over five category scores, can be tried on this study's runs. There the t-test contradicts the seed rule in both comparisons of RQ2(a). It declares the final GAN Detector better than the baseline, although on individual predictions the two differ in no consistent direction across the seeds, and it does not declare the final Detector better than the Dual-Agent system, although the Detector is correct significantly more often in every seed. With four degrees of freedom, a t-test over category scores can become significant because the scores happen to move together, and it can miss a difference that is consistent on the instances but uneven across the categories. A single run adds a second limitation, because it cannot show how far the seed alone moves the result. Neither point shows the base study's conclusions to be wrong. Together they show that its procedure could not have told the neighbouring differences of its ranking apart from those that the seed alone produces in this replication.

8.3 What the Adversarial Loop Learned

The learning signal. The loop learns from its verdicts on generated cases, so the value of that signal depends on whether the cases carry their label. The hand check indicates that they mostly do not. No sampled case carries its assigned weakness, and most of the cases the in-loop Detector let pass contain no weakness at all. Each such pass entered the memory update as a missed vulnerability, so in the sampled cases the adversarial stage mostly turned a correct *Not Vulnerable* into a supposed miss. The high pass-through rate is accordingly no sign of a Generator that deceives the Detector, but of one that mostly fails to inject a flaw. Synthetic injection is known to produce changes that do not hold (Nong et al., 2023). In this sample, that is the rule. Section 1.2 named this as the point where the transfer from news to code is not obvious, and the results locate the difficulty there and not in the mechanics of the loop, which ran as designed.

Where an effect could come from. The round curve of seed 42 suggests one interpretation, which the data support only as a possibility. The snapshot at the end of the adversarial stage is the weakest on the curve, and the self-reflection stage, which works on real labelled data, brings the Detector back to the level of round zero. On the validation probe of the same seed, the share of *Vulnerable* verdicts rises during the adversarial stage and partly falls back after reflection. This is consistent with the adversarial stage supplying a biased signal and the reflection stage correcting it. Reflection uses the only error signal in the loop that rests on true labels, an external signal of the kind Reflexion uses rather than the intrinsic feedback of Self-Refine (Madaan et al., 2023; Shinn et al., 2023). Four limits apply. The difference named in advance, between the last adversarial snapshot and the final one, is not significant, and of the ten pairwise comparisons along the curve none survives the Holm adjustment. The only nominal p below 0.05 lies between two adversarial snapshots. The curve exists for seed 42 only, and in seeds 43 and 44 the final Detector is ahead of round zero while in seed 42 it is not, so the loop may help in some runs and not in others. The probe supports the reading for seed 42 only: in seeds 43 and 44 the share of *Vulnerable* verdicts does not fall back after reflection but rises further. Beyond these limits, a single diagnostic run of the self-reflection stage alone, for seed 42 on the validation split, ended above both round zero and the final snapshot of the full loop, which is consistent with the interpretation but cannot establish it.

What the memories contain. The memories fit this picture. The final Detector memories mix statements about the labelling convention with check-

able heuristics and general advice on how to write code, and the Generator memories more often describe how to obscure code than which flaw to introduce (Section 7.6). Textual feedback can improve a prompt (Pryzant et al., 2023; Yuksekgonul et al., 2025), but only as far as it is informative, and here it concerned cases whose flaw was mostly absent. That some Detector entries restate the labelling convention links the extension to the counter-reading of Section 8.2. A loop that reflects on Big-Vul's labels can pick up Big-Vul's convention just as fine-tuning can.

The two-stage party. Placing the Dual-Agent system on the detecting side changes the course of the loop but not its outcome. The two-stage party lets fewer generated cases pass and answers more self-reflection items correctly than the single Detector, so it receives fewer updates. Its final memory lifts it above its own round zero in every seed, but it ends below the final GAN Detector in every seed as well. Reading the two stages apart shows where the difference arises (Table 7.8). Under the learned memory, the Detector stage alone reaches the level of the final GAN Detector on average, and the Validator's revision lowers it again, as in the replication. In seeds 43 and 44 the memory lifts the Detector stage above its round-zero level, in seed 42 it does not, and the Validator lowers it in every seed. Whether what the loop learns reaches the first stage is therefore suggested, not shown. Whether a two-stage party would judge better with the memory the single Detector learned was not tested, so the results cannot tell whether inferring in two stages or learning in two stages is the limiting factor.

The transfer. Measured against its template, the extension answers the transfer question negatively for this setting. LLM-GAN and SALF work on news, where the label of a generated item comes for free, and neither reports a run with a small open model. ShellForge couples generation and detection for code, but through gradient training (Ding, 2026). The present results add that once the label of generated code has to be earned rather than assumed, a prompting-only loop on a small model does not earn it, and the rest of the loop inherits this deficit.

8.4 Implications and Practical Feasibility

Three consequences for how comparisons of this kind are evaluated follow from the previous sections. On balanced data, F1 should not be reported without the predicted-positive rate and a measure that is zero for a classifier that ignores its input. Otherwise, a mere shift towards *Vulnerable* passes for a

gain (Chicco and Jurman, 2020). An order of approaches whose differences amount to a few hundredths needs repeated runs and a test on individual predictions. And an arrangement that adds a role or a loop should be reported with its cost and against the configuration it adds to, because more computation is not the same as a better mechanism (He, Treude, and Lo, 2025; Huang et al., 2024).

For someone working with a model of this size, the results point in one direction. The approach that separates best is also the cheapest to run: a short parameter-efficient training run and an answer of a few tokens, against hundreds of generated tokens per instance for every prompting approach (Section 7.5). The second agent and the adversarial loop add inference or preparation cost without a significant gain over the baseline. Whether the adversarial approach falls short because of its idea or because of the model it runs on, this study cannot settle, since it uses one model. The results locate the shortfall in roles that demand competence of that model: a Generator that has to introduce a specific flaw without breaking or merely rewriting the function, and a reflection stage whose error cases come from Detector verdicts that are near chance. These are the places where a larger model could plausibly do better, and the template reports no run with a model of this size. Scale is no assurance, however, since large models, too, are reported to identify vulnerabilities unreliably (Ullah et al., 2024). Prompt-based adversarial detection with a model of three billion parameters is thus feasible to run, since the loop stays within the context window and completes within hours, but in this study it is not effective: it does not separate the classes consistently better than direct prompting, and it does not produce the adversarial inputs its robustness question needs.

8.5 Threats to Validity

This section discusses the threats that could change a verdict of Section 8.1 or how it should be read. Some further threats limit how far individual numbers can be trusted, but change no verdict. First, Big-Vul was public before the model was trained, so the model may have seen some of its fixes (Riddell, Ni, and Cohan, 2024). This could raise all absolute values, but it would distort the comparison only if some approaches benefited from it more than others. Second, the extension uses one rule for condensing its memories and one wording for its prompts, and alternatives were not tested, so the results of RQ2 and RQ3 hold for this configuration only. Near-duplicates across the splits, the rule that turns an answer into a label and the remaining non-determinism are discussed where they are introduced (Sections 6.1, 6.4 and 6.2).

8.5.1 Construct Validity

The base study's prompt, adopted here, does not pose the task of a detector facing unreviewed code. It supplies the commit message and the CVE and CWE identifiers, which exist only once a vulnerability has been fixed, and it shows the function before and after the fix, whose identity determines the label on the test split (Sections 3.4 and 8.2). No approach comes near exploiting that shortcut, so it does not decide the comparison, but all absolute values belong to a task whose answer is contained in its input, and fine-tuning may partly learn the convention rather than the task (Section 8.2). Big-Vul derives its labels from fixing commits, and labels of this kind have been found inaccurate (Croft, Babar, and Kholoosi, 2023). Label noise lowers the attainable level of every approach, but it favours no approach at evaluation time. Approaches that learn from the labels, the fine-tuned configuration and the reflection stage, may in addition absorb the convention behind them (Sections 8.2 and 8.3).

The verdict on RQ2(b) and the interpretation of the loop rest on one sample from one run, rated by one person, not blind, after a preliminary rating by the tooling assistant, so the ratings may be anchored. The confidence interval covers sampling error only. Misrating is a separate risk: for the finding to change direction, a substantial share of the 30 cases would have to be misrated, and only six of them were rated as carrying any weakness at all. The extension's arrangement is self-play, not an attacker model, and its generated case is an isolated function next to its clean original, the framing whose limits Risse, Liu, and Böhme (2025) point out and a pair that detectors separate poorly (Risse and Böhme, 2024).

8.5.2 Internal Validity

The SFT adapter is optimised against the 4-bit quantised backbone but scored merged into the shared fp16 weights, and the training library differs from the authors' (Section 4.4). Neither departure was measured. For the order of RQ1 they matter only if their effect exceeds the margin between SFT and RAG, which is 0.153 in mean macro-F1. For the advantage of SFT over the baseline, whose significance is narrow, a smaller effect could suffice. The decoding of the RAG approach was fixed after a first test run of seed 42 under the released decoding had been seen (Section 4.3). That run reached a macro-F1 of 0.481, below the baseline of the same seed, against 0.561 under the shared decoding, so the direction of the F1 difference between RAG and the baseline depends on the decoding. So does the test. Under the released decoding, RAG was significantly less often correct than the baseline in that seed, whereas under the shared decoding the pair is unresolved. A defect confined to one approach

would bias the comparison. Beyond the test suite, three checks on the official runs speak against it: the round-zero snapshot of the GAN loop and the Detector pass of the Dual-Agent system each reproduce the baseline run of the same seed text for text, and the round-zero party of the Triple-Agent loop reproduces the Dual-Agent run in the same way.

8.5.3 Statistical Conclusion Validity

McNemar's test decides on correctness, while the order of RQ1 is defined by macro-F1. Where the orders part, as for RAG and the baseline in one seed, Section 7.2.1 marks it. The largest p-value over three seeds, with the same direction required in each, is conservative, so the tests have little power for differences of a few hundredths. That the baseline is not significantly ahead of the Dual-Agent system, and that RAG and the baseline do not differ significantly, are absences of evidence, not evidence of equality. The one verdict that depends on how a family is read is the advantage of SFT over the baseline, which holds for the conjunction fixed in Section 6.5 and, under the primary policy, not under Holm.

8.5.4 External Validity

All results come from one corpus, the C and C++ functions of Big-Vul in the five categories of the base study, balanced per category, and from one model of one size. Scores on Big-Vul have not carried over to a deduplicated, chronologically split dataset (Ding et al., 2025), and on realistic data, where vulnerable functions are rare, precision in particular falls away from what balanced sets suggest (Chakraborty, Arumugam, et al., 2024). The absolute values say nothing about practical detection, and the order is established for Big-Vul only. With another corpus or model it may differ, most plausibly for RAG, whose effect depends on its knowledge base, and for the extension, whose roles depend on the competence of the model.

Chapter 9

Conclusion and Future Work

9.1 Summary of Contributions

This thesis set out from a comparison of four LLM-based approaches to code vulnerability detection whose reported ordering rests partly on data that cannot be obtained, and from the absence of an adversarial signal among those approaches. It addressed both with the same model, the same public corpus and one evaluation pipeline.

Replication. Baseline, RAG, SFT and the Dual-Agent system were reimplemented independently and run on Big-Vul alone, each under three random seeds, with every comparison paired by seed and decided by McNemar's test. The reported order does not hold. In every seed the fine-tuned configuration leads RAG, significantly and against the base study's order, and the Dual-Agent system falls behind the baseline, though not significantly. Of the three interventions, only fine-tuning outperforms the baseline significantly, and under the primary policy that result does not survive the Holm adjustment. Measured by the Matthews correlation, RAG and the adversarial Detector differ from the baseline mainly in how often they answer *Vulnerable*, not in how well they separate the classes, while the Dual-Agent system separates barely better than chance. Fine-tuning is the exception, and part of what it learns may be the labelling convention of the corpus. Because the absolute values rest on a corpus without the proprietary portion and on conditions of the original runs that are not public, the result speaks to the ordering and not to the level.

Adversarial extension. The central result of the extension is negative and concerns its premise. The label that the loop attributes to generated functions was checked rather than assumed, and none of 30 sampled functions from the loop of seed 42 carries the assigned weakness, and most carry no weakness at

all. The transfer itself was carried out with both stages of LLM-GAN, namely an adversarial loop in which Generator and Detector exchange text rather than gradients, and a reflection stage on real labelled data. Against the two approaches that likewise use prompting alone, the final Detector is correct significantly more often than the Dual-Agent system, a comparison that largely reflects that system's second stage, but not more often than the baseline. Because the generated cases lack the intended flaw, neither comparison can be read as the effect of adversarial learning, and robustness could not be measured as posed. Employing the full Dual-Agent system as the detecting party (RQ3) did not improve on the adversarial setup. The final Triple-Agent party is correct less often than the final GAN Detector in every seed, though not significantly, and its first stage alone reaches the level of the single Detector before the Validator's revision lowers it again.

Reproducibility audit. The published record of the base study was inventoried parameter by parameter (Section 4.6). The release and the authors' answers closed mostly configuration values. What remains open or unavailable concerns the data and the conditions of the reported runs: the proprietary portion of the corpus, the curation behind the splits, the expert review of the labels, the documents behind the RAG knowledge base, and the serving of the original runs. These are the parts a public replication cannot substitute, and they bound what stays comparable.

With a model of three billion parameters, prompt-based adversarial detection can be run but was not effective in this study. It neither separated the classes consistently better than direct prompting nor produced the adversarial inputs its robustness question needs.

9.2 Future Work

Checks within reach. Three runs lie outside the questions of this thesis but would sharpen the reading of its results, and the existing code supports them. The SFT adapter was trained for the step count of the base study, which covers only a fraction of one epoch. A run over a full epoch would show whether the approach is under-trained. For the adversarial loop, the self-reflection stage alone was run once, for seed 42 on the validation split. The same run on the test split and for all three seeds would test the interpretation of the round curve in Section 8.3. And evaluating the strategy memory learned by the RQ2 loop with the two-stage detecting party would separate inferring in two stages from learning in two stages, which the Triple-Agent runs leave confounded. One further check does need new code, a quantised inference path: evaluating

the SFT adapter in the quantised state it was trained in would remove one of the departures named in Section 8.5.2.

A firmer ground truth for generated code. The label of a generated function was checked by one rater, not blind, on a sample. Several independent raters with an agreement statistic would turn that check into a measurement with a known error. A stronger step needs the context that the isolated function bodies of Big-Vul lack. Placed back into its project at the commit it stems from, a generated function could be built and exercised under a sanitizer, which would give execution-based evidence of the intended flaw. Only generated functions with such evidence would make the robustness question of RQ2(b) measurable.

Generalisation and the task as posed. The order of the approaches is established for one corpus and one model. Repeating the comparison on PrimeVul, whose stricter labels, deduplication and chronological split remove much of what inflates scores on Big-Vul (Section 3.4), would show whether the order survives more realistic data. Larger models are the most likely way to change the picture. The extension fails in roles that demand competence of the model, a Generator that has to inject a specific flaw and a reflection stage fed by Detector verdicts that are close to chance (Section 8.4), the Validator makes the Detector's verdicts worse, and knowledge-level retrieval has helped larger models separate vulnerable from patched code (Du et al., 2025). Repeating the comparison with larger models would show which of these failures are failures of model competence. The prompt of the base study supplies the commit message, the CVE and CWE identifiers and the fixed version of the function, none of which a detector deployed on unreviewed code would have (Section 8.5.1). The same comparison without these fields is the most direct continuation of this work. A baseline of matched inference cost, such as repeated sampling with a majority vote (Huang et al., 2024), would show how much of an advantage is additional computation. Finally, only verdicts were evaluated. Whether the accompanying justifications are correct is a question of its own.

Appendix A

Full Prompts

A.1 Role Prompts of the Extension

The six texts below are the self-authored prompts of Chapter 5 (Section 5.1), printed as the model received them, with the line breaks of the source file except where a placeholder lengthens a line. Text in double braces, `{{...}}`, is inserted at run time and marks a record field, a model output or a strategy memory. Angle brackets are part of the prompt itself. A strategy memory is rendered as one numbered line per entry, and the blocks introducing it are omitted while the memory is empty. The numbers in the update, reflection and condensation prompts, at most 2 new entries, at most 30 words and at most 12 entries, are the values of Section 5.3. The Figure 1 prompt of the base study is not reproduced here. It enters the Detector texts unchanged (Section 5.3).

Generator (`gan-generator-v1`). The generating call of the adversarial stage (Section 5.2).

```
You are an expert in offensive software security, acting as an adversarial code
generator. Below is a C/C++ function that does not currently contain the weakness
class named in the metadata. Write a modified version of that function which DOES
contain an instance of that weakness class, and make the flaw as hard to notice as
possible for an automated code reviewer.

Requirements:
1) Keep the function's name, signature, and overall purpose unchanged. The modified
   function must still read like ordinary production code of this project.
2) Introduce exactly one instance of the weakness class given below. The flaw must be
   real: there must be an input or program state under which the modified function
   behaves unsafely.
3) Do not add comments, identifiers, or markers that hint at the flaw, and do not
   mention the weakness anywhere in the code.
4) Keep the change small. A reviewer diffing your version against the original should
   see a plausible, unremarkable modification.
5) Also write a commit message of the kind a developer would write for this change. It
```

```
   must present the change as routine work (refactoring, cleanup, performance, feature)
   and must not mention security, vulnerabilities, or the weakness.

Strategies you compiled in earlier rounds. Apply the ones that fit this function:
{{S_G}}

Here is the data:
- Project: {{project}}
- Programming Language: {{language}}
- Weakness class to introduce: {{CWE ID (official CWE name)}}
- Function:
{{pre-commit function}}

Reply in exactly this format, with nothing before or after it:
=== COMMIT MESSAGE ===
<one or two lines>
=== MODIFIED FUNCTION ===
<the complete modified function, code only, no markdown fences>
=== DECEPTION NOTES ===
<two to four sentences: what you changed, why it is unsafe, and what makes the flaw
hard to notice>
```

Generator update (`gan-strategy-update-v1`)**.** Issued after a generated function was detected. The Generator extends S_G itself.

```
You are an expert in offensive software security, maintaining a written playbook of
strategies for hiding vulnerabilities in code so that automated reviewers miss them.

In the last round your modified function was correctly identified as vulnerable. The
reviewer's own assessment below states how it reached that conclusion. Work out what
your playbook is still missing.

Your current playbook:
{{S_G}}

--- The original function ---
{{pre-commit function}}

--- Your modified function ---
{{generated function}}

--- Your notes on how you hid the flaw ---
{{deception notes of the generating call}}

--- The reviewer's assessment ---
{{Detector output}}

Write at most 2 new playbook entries. Each entry must be a single imperative
sentence of at most 30 words, must generalise to other functions (do not mention this
function, its identifiers, or this project), and must not repeat an entry already in
the list. If nothing new can be learned, write NONE.

=== NEW STRATEGIES ===
- <entry>
```

Detector block (`gan-detector-block-v1`)**.** Prepended to the unchanged Figure 1 prompt. With an empty memory the Detector prompt is the baseline prompt, byte for byte (Section 5.3). In the Triple-Agent party the same block also precedes the authors' Validator prompt (Section 5.6).

```
You have compiled the following detection strategies from earlier cases. Consider the
ones that apply to the case below, but base your verdict on the evidence in the data -
a strategy is a hint, not a verdict.
{{S_D}}
---

{{Figure 1 prompt of the instance, unchanged}}
```

Detector update (`gan-strategy-update-v1`)**.** One template with two evidence variants: after a missed generated function the evidence is the Generator's deception notes, after an error on a training record it is the Reflector's output (Section 5.3). In the Triple-Agent party the assessment is the two-stage transcript, whose stages are labelled `Detector (first agent)` and `Validator (second agent, final verdict)`.

```
You are an expert in cybersecurity and software vulnerability detection, maintaining a
written list of detection strategies: the lessons you apply when judging whether code
contains a vulnerability.

You judged the case below incorrectly.
{{adversarial variant:}}
The code was deliberately modified to contain a vulnerability, and the notes from its
author explain how the flaw was hidden from you.
{{reflection variant:}}
The reflection below explains where your reasoning went wrong and what the data
actually shows.

Your current strategies:
{{S_D}}

--- The case as it was presented to you ---
{{Figure 1 prompt of the instance}}

--- Your assessment ---
{{Detector output}}

--- How the flaw was hidden ---            {{adversarial variant}}
{{deception notes}}
--- Reflection on your mistake ---         {{reflection variant}}
{{Reflector output}}

Write at most 2 new strategy entries. Each entry must be a single imperative
sentence of at most 30 words, must generalise to other cases (do not mention this
function, its identifiers, or this project), and must not repeat an entry already in
the list. If nothing new can be learned, write NONE.

=== NEW STRATEGIES ===
- <entry>
```

Reflector (`gan-reflector-v1`). Called on a training record the Detector judged wrongly. The Reflector is told the gold label (Section 5.3). Where the Detector's output yielded no label, the analyst's answer is given as `no clear verdict`.

```
You are a senior security reviewer acting as a reflector. A junior analyst was given
the task below and answered "{{parsed label}}", but the correct answer is
"{{gold label}}". Explain the mistake so that the analyst can learn from it. Do not be
vague: name the specific step in the analyst's reasoning that failed.

--- The task given to the analyst (including the data) ---
{{Figure 1 prompt of the instance}}

--- The analyst's assessment ---
{{Detector output}}

Answer in exactly this format, with nothing before or after it:
=== WHERE THE ANALYSIS WENT WRONG ===
<which claim or assumption in the assessment is unsupported or wrong>
=== EVIDENCE FOR THE CORRECT ANSWER ===
<what in the provided data supports "{{gold label}}" - refer to concrete code, the
commit message, or the CWE information>
=== LESSON ===
<one imperative sentence of at most 30 words that generalises to other cases>
```

Memory condensation (`gan-summarize-v1`). Issued when a memory exceeds its cap of 12 entries under the policy `summarize` (Section 5.3). It is not part of the source framework.

```
Condense the following list of {{n}} {{detector | generator}} strategies into at most
12 entries without losing any distinct lesson. Merge entries that say the same thing,
drop entries that are special cases of another entry, and keep the wording imperative
and at most 30 words per entry.

Current strategies:
{{the entries, numbered}}

=== NEW STRATEGIES ===
- <entry>
```

A.2 Final Strategy Memories

This section gives the final memories of the GAN loop run of seed 42, the seed of the hand check and of the round curve, both S_D and S_G (Section 5.3), verbatim except for the markdown emphasis of the entry names, which is set in italics. The memories of seeds 43 and 44 and of the Triple-Agent runs are in the repository.

GAN, Detector strategies S_D, seed 42.

1. Simplify conditional statements carefully to avoid introducing new vulnerabilities.
2. Validate and sanitize variables before reusing them in loops and conditional statements.

3. Preserve critical checks in conditional statements to prevent introducing new vulnerabilities.
4. Validate and sanitize input before using flags or variables to prevent malicious input injection.
5. Reuse flags and variables with caution, resetting them properly to prevent exploitation of previous states.
6. Verify the initialization of member variables and pointers to prevent null pointer dereferences.
7. Use meaningful variable names to avoid misleading code reviews and make vulnerabilities more apparent.
8. Document security-critical logic and assumptions to make vulnerabilities more apparent.
9. Verify the bounds of array or buffer operations to prevent potential buffer overflow vulnerabilities.
10. Review surrounding code thoroughly before making changes to ensure no new vulnerabilities are introduced.
11. Ensure fixes address the root cause of vulnerabilities, not just symptoms, to prevent re-introduction of issues.
12. Verify code changes appear identical before and after to ensure no new vulnerabilities are introduced.

GAN, Generator strategies S_G, seed 42.

1. *Disable Security Checks*: Disable security checks to create backdoors or introduce vulnerabilities.
2. *Obfuscate Code*: Obfuscate code with complex logic to hide vulnerabilities or malicious code.
3. *Misuse Functionality*: Misuse a feature to bypass security checks or exploit code vulnerabilities.
4. *Indirect Function Calls*: Use indirect function calls to obscure code flow and hide vulnerabilities.
5. *Variable Order Manipulation*: Rearrange variable declarations to obscure code flow and potential vulnerabilities.
6. *Truncate Data*: Truncate data in mid-operation to create a temporary, usable piece that can be modified later.
7. *Deceptive Functionality*: Introduce inconsistent or contradictory behavior to mislead automated security tools.
8. *Silence Error Handling*: Silence error handling statements for exceptional error cases to hide issues.
9. *Manipulate Buffer Sizes*: Omit or manipulate buffer size checks to introduce vulnerabilities or obscure code flow.
10. *Misleading Assumptions*: Make assumptions about function parameters to lead to incorrect or incomplete error handling.

11. *Use Context-Specific Type Casting*: Use explicit type casting to obscure variable values and make vulnerabilities harder to detect.
12. *Bypass Security Checks*: Use temporary flags, indirect function calls, or other means to bypass security checks and introduce vulnerabilities.

Appendix B

Author Correspondence

The questions below were sent to the authors of the base study on 7 August 2026. A reply from M. Muhtadi, with M. H. Saju in copy, followed on 10 August 2026. The request is reproduced verbatim. Only the numbering of the nine questions has been added, so that Table B.1 can refer to them, and the typography has been adapted to the typesetting of this thesis. The answers are paraphrased throughout, because no permission to quote them was obtained. The table records, for each question, whether it was answered, answered in part, or not answered, and what the answer amounted to. Where the body of the thesis relies on one of these answers, it cites this appendix.

The request

> Dear Hasan Saju, Maher Muhtadi and Akramul Azim
>
> My name is Elias Masur. I am a computer science student at Leipzig University, Germany, and for my bachelor's thesis (supervised by Prof. Dr. Andreas Maletti) I am replicating and extending the four approaches from your paper "An Empirical Evaluation of LLM-Based Approaches for Code Vulnerability Detection: RAG, SFT, and Dual-Agent Systems" (CASCON 2025, arXiv:2601.00254). I think it is a very useful comparison, and I would like to reproduce your results as faithfully as possible.
>
> I have implemented everything the paper specifies: Big-Vul as the data source, the five CWE categories with per-CWE class balance, the Fig. 1 prompt verbatim, the same prompt across all approaches, the Table I fine-tuning configuration, and the per-CWE / All (Avg) metrics with the paired t-tests. However, footnote 1 refers to `github.com/Hasan-Saju/llm-code-vulnerability` for the training setup, hyperparameters, prompts, and data, and the repository currently contains only a README. Since my baseline results diverge notably from Table V

(much lower recall — my model answers "Not Vulnerable" for about 80% of instances), I would like to rule out that the divergence comes from details I could not reconstruct from the paper. I would be very grateful if you could help me with the following questions:

1. Model variant and checkpoint. Was "LLaMA 3.2-3B" the base model or the Instruct variant, and which exact checkpoint (e.g., the Hugging Face repo id)?
2. Serving stack and decoding. Which framework served the model (HF transformers, Ollama, llama.cpp, Unsloth, . . .), at which precision/quantization on the 15-GB T4, and with which decoding parameters (temperature, top-p, max new tokens; greedy or sampling)? Was the Fig. 1 prompt passed as a system or a user message, and was any additional system prompt involved?
3. Output-to-label rule. How was the free-text answer reduced to Vulnerable / Not Vulnerable? How did you count outputs that contain neither answer word, hedge, or refuse — and were any outputs excluded?
4. Dataset composition and curation. Could you share the curated dataset, or at least the Big-Vul instance identifiers (commit_ids) of the 5000 training and 1000 test instances? What was the Big-Vul : GlassHouse ratio, and did GlassHouse instances appear in the test set? Were very long functions truncated or excluded, and if so, at what maximum input length?
5. CVE ID field. The feature list in §III.A does not include a 'CVE ID' field, but the Fig. 1 prompt reads `sample.get('CVE ID', 'N/A')` — did your prompts contain actual CVE IDs, or did this field always render as "N/A"?
6. Train/test split. Was the split random at the instance level, or grouped (by commit or project)? Was leakage between train and test controlled?
7. Metric conventions. Which class is the positive class for precision/recall/F1, and is "All (Avg)" the unweighted mean of the five per-CWE rows?
8. Repository. Is the material referenced in footnote 1 available elsewhere, or would it be possible to push it to the repository? This single item would resolve most of the questions above.
9. Seeds and repetitions. Were the reported numbers from a single run or averaged over repetitions, and was a random seed fixed?

Even partial answers — or just the repository contents (question 8) — would help me a lot. I will of course cite your paper as the basis of my

thesis, and I would be happy to share my replication results with you if they are of interest.

Thank you very much for your time, and congratulations on the paper.

Best regards,
Elias Masur
Leipzig University

The answers

Of the nine questions, three were answered in full, five in part, and one not at all (Table B.1). Two gaps weigh most for the replication. The curated dataset was withheld, and with it the instance identifiers and the mixing ratio. Together with the confirmation that the proprietary portion reaches the test set, this rules out an exact reproduction of the reported numbers (Section 4.1). The conditions under which the reported numbers were produced also remain open. The answer names serving stacks but no precision and no decoding parameters, leaves the RAG approach out, and places two approaches on a hosted service where the paper reports a single T4 GPU for all experiments (Section 4.6). The split procedure, the one question left unanswered, bears on how the original numbers are read rather than on this replication, which builds its own splits.

Table B.1: Questions put to the authors and the state of each answer. Answers are paraphrased. *Partial* means that the question was addressed but a part of what it asked for was not supplied.

No.	Topic	Status	Answer, paraphrased
1	Model variant, checkpoint	partial	The Instruct variant was used, fine-tuned with Unsloth. No checkpoint or revision was named.
2	Serving stack, decoding	partial	Together AI served the base models and the Dual-Agent prompting, where some models had since been discontinued and replaced by alternatives. The fine-tuning ran through Unsloth. The RAG approach is not covered by this assignment, and no inference precision, no decoding parameters, and nothing on the system or user placement of the prompt were given.

continued on next page

Table B.1, continued

No.	Topic	Status	Answer, paraphrased
3	Output-to-label rule	answered	String matching on the two answer words. Outputs matching neither were dropped from the metrics.
4	Dataset, curation	partial	The proprietary portion is present in both the training and the test set, and for that reason the curated dataset was not shared. No instance identifiers, no mixing ratio, and nothing on truncation were provided.
5	CVE ID field	answered	The prompts contained actual CVE identifiers. Instances whose CVE was not confirmed carried the placeholder “none”.
6	Split, leakage	unanswered	Only a general statement that both splits hold sufficient representation of all groups, and nothing on grouping level or leakage control.
7	Metric conventions	answered	“Vulnerable” is the positive class. The overall row is the unweighted mean over the five categories.
8	Repository	partial	The answer referred to only part of the code blocks and stated that different prompts had been used for the experiments. An inspection of the repository after the reply showed that the push comprised a notebook, the Dual-Agent prompt pair and the retrieval code. Data and the documents behind the knowledge base were not released.
9	Seeds, repetitions	partial	Each configuration was run once. No seed was named.

Bibliography

Baltes, Sebastian, Florian Angermeir, Chetan Arora, Marvin Muñoz Barón, Chunyang Chen, Lukas Böhme, Fabio Calefato, Neil Ernst, Davide Falessi, Brian Fitzgerald, Davide Fucci, Junda He, Christoph Treude, Marcos Kalinowski, Stefano Lambiase, Daniel Russo, Mircea Lungu, Cristina Martinez Montes, Lutz Prechelt, Paul Ralph, Rijnard van Tonder, and Stefan Wagner (2026). *Guidelines for Empirical Studies in Software Engineering Involving Large Language Models*. arXiv: 2508.15503 [cs.SE]. Preprint.

Berger, Roger L. (1982). "Multiparameter Hypothesis Testing and Acceptance Sampling". In: *Technometrics* 24.4, pp. 295–300. DOI: 10.2307/1267823.

Chakraborty, Partha, Krishna Kanth Arumugam, Mahmoud Alfadel, Meiyappan Nagappan, and Shane McIntosh (2024). "Revisiting the Performance of Deep Learning-Based Vulnerability Detection on Realistic Datasets". In: *IEEE Transactions on Software Engineering* 50.8, pp. 2163–2177. DOI: 10.1109/TSE.2024.3423712.

Chakraborty, Saikat, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray (2022). "Deep Learning Based Vulnerability Detection: Are We There Yet?" In: *IEEE Transactions on Software Engineering* 48.9, pp. 3280–3296. DOI: 10.1109/TSE.2021.3087402.

Chen, Yizheng, Zhoujie Ding, Lamya Alowain, Xinyun Chen, and David Wagner (2023). "DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection". In: *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*. Association for Computing Machinery, pp. 654–668. DOI: 10.1145/3607199.3607242.

Chicco, Davide and Giuseppe Jurman (2020). "The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation". In: *BMC Genomics* 21.1, p. 6. DOI: 10.1186/s12864-019-6413-7.

Croft, Roland, M. Ali Babar, and M. Mehdi Kholoosi (2023). "Data Quality for Software Vulnerability Datasets". In: *Proceedings of the 45th International Conference on Software Engineering*. IEEE Press, pp. 121–133. DOI: 10.1109/ICSE48619.2023.00022.

Dettmers, Tim, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer (2023). *QLoRA: Efficient Finetuning of Quantized LLMs*. arXiv: 2305.14314 [cs.LG]. Preprint.

Dietterich, Thomas G. (1998). "Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms". In: *Neural Computation* 10.7, pp. 1895–1923. DOI: 10.1162/089976698300017197.

Ding, Yangruibo, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen (2025). "Vulnerability Detection with Code Language Models: How Far Are We?" In: *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, pp. 1729–1741. DOI: 10.1109/ICSE55347.2025.00038.

Ding, Yizhong (2026). *ShellForge: Adversarial Co-Evolution of Webshell Generation and Multi-View Detection for Robust Webshell Defense*. arXiv: 2601.22182 [cs.CR]. Preprint.

Du, Xueying, Geng Zheng, Kaixin Wang, Yi Zou, Yujia Wang, Wentai Deng, Jiayi Feng, Mingwei Liu, Bihuan Chen, Xin Peng, Tao Ma, and Yiling Lou (2025). *Vul-RAG: Enhancing LLM-Based Vulnerability Detection via Knowledge-Level RAG*. arXiv: 2406.11147 [cs.SE]. Preprint.

Durumeric, Zakir, Frank Li, James Kasten, Johanna Amann, Jethro Beekman, Mathias Payer, Nicolas Weaver, David Adrian, Vern Paxson, Michael Bailey, and J. Alex Halderman (2014). "The Matter of Heartbleed". In: *Proceedings of the 2014 Conference on Internet Measurement Conference*. Association for Computing Machinery, pp. 475–488. DOI: 10.1145/2663716.2663755.

Fagerland, Morten W., Stian Lydersen, and Petter Laake (2013). "The McNemar Test for Binary Matched-Pairs Data: Mid-p and Asymptotic Are Better than Exact Conditional". In: *BMC Medical Research Methodology* 13.1, p. 91. DOI: 10.1186/1471-2288-13-91.

Fan, Jiahao, Yi Li, Shaohua Wang, and Tien N. Nguyen (2020). "A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries". In: *Proceedings of the 17th International Conference on Mining Software*

Repositories. Association for Computing Machinery, pp. 508–512. DOI: 10.1145/3379597.3387501.

Goodfellow, Ian J., Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio (2014). "Generative Adversarial Nets". In: *Advances in Neural Information Processing Systems*. Vol. 27. Curran Associates, Inc. URL: https://papers.nips.cc/paper_files/paper/2014/hash/f033ed80deb0234979a61f95710dbe25-Abstract.html.

Gou, Zhibin, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen (2024). "CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing". In: *International Conference on Learning Representations*. URL: https://openreview.net/forum?id=Sx038qxjek.

Grattafiori, Aaron et al. (2024). *The Llama 3 Herd of Models*. arXiv: 2407.21783 [cs.AI]. Preprint.

He, Junda, Christoph Treude, and David Lo (2025). "LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead". In: *ACM Transactions on Software Engineering and Methodology* 34.5, 124:1–124:30. DOI: 10.1145/3712003.

Holm, Sture (1979). "A Simple Sequentially Rejective Multiple Test Procedure". In: *Scandinavian Journal of Statistics* 6.2, pp. 65–70. URL: https://www.jstor.org/stable/4615733.

Holtzman, Ari, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi (2020). "The Curious Case of Neural Text Degeneration". In: *International Conference on Learning Representations*. arXiv: 1904.09751 [cs.CL].

Hu, Edward J., Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen (2022). "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations*. arXiv: 2106.09685 [cs.CL].

Hu, Sihao, Tiansheng Huang, Fatih İlhan, Selim Furkan Tekin, and Ling Liu (2023). *Large Language Model-Powered Smart Contract Vulnerability Detection: New Perspectives*. arXiv: 2310.01152 [cs.CR]. Preprint.

Huang, Jie, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou (2024). *Large Language Models Cannot Self-Correct Reasoning Yet*. arXiv: 2310.01798 [cs.CL]. Preprint.

Johnson, Brittany, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge (2013). "Why Don't Software Developers Use Static Analysis Tools to Find Bugs?" In: *Proceedings of the 2013 International Conference on Software Engineering*. IEEE Press, pp. 672–681. URL: https://dl.acm.org/doi/10.5555/2486788.2486877.

Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems*. Vol. 33. Curran Associates, Inc., pp. 9459–9474. URL: https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

Lipp, Stephan, Sebastian Banescu, and Alexander Pretschner (2022). "An Empirical Study on the Effectiveness of Static C Code Analyzers for Vulnerability Detection". In: *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. Association for Computing Machinery, pp. 544–555. DOI: 10.1145/3533767.3534380.

Liu, Chao, Cuiyun Gao, Xin Xia, David Lo, John Grundy, and Xiaohu Yang (2022). "On the Reproducibility and Replicability of Deep Learning in Software Engineering". In: *ACM Transactions on Software Engineering and Methodology* 31.1, 15:1–15:46. DOI: 10.1145/3477535.

Liu, Nelson F., Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang (2024). "Lost in the Middle: How Language Models Use Long Contexts". In: *Transactions of the Association for Computational Linguistics* 12, pp. 157–173. DOI: 10.1162/tacl_a_00638.

Liu, Shigang, Di Cao, Junae Kim, Tamas Abraham, Paul Montague, Seyit Camtepe, Jun Zhang, and Yang Xiang (2024). "EaTVul: ChatGPT-Based Evasion Attack Against Software Vulnerability Detection". In: *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, pp. 7357–7374. URL: https://www.usenix.org/conference/usenixsecurity24/presentation/liu-shigang.

Madaan, Aman, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark (2023). *Self-Refine: Iterative Refinement with Self-Feedback*. arXiv: 2303.17651 [cs.CL]. Preprint.

Mao, Zhenyu, Jialong Li, Dongming Jin, Munan Li, and Kenji Tei (2024). *Multi-Role Consensus through LLMs Discussions for Vulnerability Detection*. arXiv: 2403.14274 [cs.SE]. Preprint.

McNemar, Quinn (1947). "Note on the Sampling Error of the Difference between Correlated Proportions or Percentages". In: *Psychometrika* 12.2, pp. 153–157. DOI: 10.1007/BF02295996.

Meta (2024). *Llama-3.2-3B-Instruct Model Card*. URL: https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct. Accessed September 8, 2026.

MITRE (2026). *Common Weakness Enumeration (CWE), Version 4.20*. CWE. URL: https://cwe.mitre.org/data/archive.html. Accessed September 8, 2026.

National Institute of Standards and Technology (2026). *NIST Updates NVD Operations to Address Record CVE Growth*. NIST. URL: https://www.nist.gov/news-events/news/2026/04/nist-updates-nvd-operations-address-record-cve-growth. Accessed September 8, 2026.

Nogueira, Rodrigo and Kyunghyun Cho (2020). *Passage Re-Ranking with BERT*. arXiv: 1901.04085 [cs.IR]. Preprint.

Nong, Yu, Yuzhe Ou, Michael Pradel, Feng Chen, and Haipeng Cai (2023). "VULGEN: Realistic Vulnerability Generation via Pattern Mining and Deep Learning". In: *Proceedings of the 45th International Conference on Software Engineering*. IEEE Press, pp. 2527–2539. DOI: 10.1109/ICSE48619.2023.00211.

Ouyang, Long, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe (2022). "Training Language Models to Follow Instructions with Human Feedback". In: *Advances in Neural Information Processing Systems*. Vol. 35. Curran Associates, Inc., pp. 27730–27744. DOI: 10.52202/068431-2011.

Pryzant, Reid, Dan Iter, Jerry Li, Yin Lee, Chenguang Zhu, and Michael Zeng (2023). "Automatic Prompt Optimization with "Gradient Descent" and Beam Search". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Ed. by Houda Bouamor, Juan Pino, and Kalika Bali. Association for Computational Linguistics, pp. 7957–7968. DOI: 10.18653/v1/2023.emnlp-main.494.

Radford, Alec, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever (2019). *Language Models Are Unsupervised Multitask Learners*. OpenAI. URL: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

Ram, Ori, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham (2023). "In-Context Retrieval-Augmented Language Models". In: *Transactions of the Association for Computational Linguistics* 11, pp. 1316–1331. DOI: 10.1162/tacl_a_00605.

Riddell, Martin, Ansong Ni, and Arman Cohan (2024). "Quantifying Contamination in Evaluating Code Generation Capabilities of Language Models". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Lun-Wei Ku, Andre Martins, and Vivek Srikumar. Association for Computational Linguistics, pp. 14116–14137. DOI: 10.18653/v1/2024.acl-long.761.

Risse, Niklas and Marcel Böhme (2024). "Uncovering the Limits of Machine Learning for Automatic Vulnerability Detection". In: *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, pp. 4247–4264. URL: https://www.usenix.org/conference/usenixsecurity24/presentation/risse.

Risse, Niklas, Jing Liu, and Marcel Böhme (2025). "Top Score on the Wrong Exam: On Benchmarking in Machine Learning for Vulnerability Detection". In: *Proceedings of the ACM on Software Engineering* 2 (ISSTA), pp. 388–410. DOI: 10.1145/3728887.

Saju, Md Hasan, Maher Muhtadi, and Akramul Azim (2026). *An Empirical Evaluation of LLM-Based Approaches for Code Vulnerability Detection: RAG, SFT, and Dual-Agent Systems*. arXiv: 2601.00254 [cs.SE]. Preprint.

Sclar, Melanie, Yejin Choi, Yulia Tsvetkov, and Alane Suhr (2024). *Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I Learned to Start Worrying about Prompt Formatting*. arXiv: 2310.11324 [cs.CL]. Preprint.

Sheng, Ze, Zhicheng Chen, Shuning Gu, Heqing Huang, Guofei Gu, and Jeff Huang (2026). "LLMs in Software Security: A Survey of Vulnerability Detection Techniques and Insights". In: *ACM Computing Surveys* 58.5, 134:1–134:35. DOI: 10.1145/3769082.

Shinn, Noah, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning.* arXiv: 2303.11366 [cs.AI]. Preprint.

Tian, Chong, Qirong Ho, and Xiuying Chen (2025). *A Symbolic Adversarial Learning Framework for Evolving Fake News Generation and Detection.* arXiv: 2508.19633 [cs.CL]. Preprint.

Ullah, Saad, Mingji Han, Saurabh Pujar, Hammond Pearce, Ayse Coskun, and Gianluca Stringhini (2024). "LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks". In: *2024 IEEE Symposium on Security and Privacy (SP).* IEEE, pp. 862–880. DOI: 10.1109/SP54263.2024.00210.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin (2017). "Attention Is All You Need". In: *Proceedings of the 31st International Conference on Neural Information Processing Systems.* Curran Associates, Inc., pp. 6000–6010. URL: https://dl.acm.org/doi/10.5555/3295222.3295349.

von Werra, Leandro, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec (2026). *TRL: Transformers Reinforcement Learning.* Version 1.9.0. URL: https://github.com/huggingface/trl. Accessed September 8, 2026.

Wang, Yifeng, Zhouhong Gu, Siwei Zhang, Suhang Zheng, Tao Wang, Tianyu Li, Hongwei Feng, and Yanghua Xiao (2025). "LLM-GAN: Constructing Generative Adversarial Network Through Large Language Models for Explainable Fake News Detection". In: *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP).* IEEE, pp. 1–5. DOI: 10.1109/ICASSP49660.2025.10889048.

Widyasari, Ratnadira, Martin Weyssow, Ivana Clairine Irsan, Han Wei Ang, Frank Liauw, Eng Lieh Ouh, Lwin Khin Shar, Hong Jin Kang, and David Lo (2026). "Let the Trial Begin: A Mock-Court Approach to Vulnerability Detection Using LLM-Based Agents". In: *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering.* Association for Computing Machinery, pp. 1030–1042. DOI: 10.1145/3744916.3773256.

Xiao, Shitao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie (2024). "C-Pack: Packed Resources for General Chinese Embeddings". In: *Proceedings of the 47th International ACM SIGIR Confer-*

ence on Research and Development in Information Retrieval. Association for Computing Machinery, pp. 641–649. DOI: 10.1145/3626772.3657878.

Yang, Zhou, Jieke Shi, Junda He, and David Lo (2022). "Natural Attack for Pre-Trained Models of Code". In: *Proceedings of the 44th International Conference on Software Engineering*. Association for Computing Machinery, pp. 1482–1493. DOI: 10.1145/3510003.3510146.

Yuksekgonul, Mert, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou (2025). "Optimizing Generative AI by Backpropagating Language Model Feedback". In: *Nature* 639.8055, pp. 609–616. DOI: 10.1038/s41586-025-08661-4.